Container-based Virtualization for Real-time Industrial Systems—A Systematic Review

RUI QUEIROZ and TIAGO CRUZ, University of Coimbra, CISUC, Department of Informatics Engineering, Portugal
JÉRÔME MENDES, University of Coimbra, CEMMPRE, ISR, ARISE, Department of Mechanical Engineering, Portugal
PEDRO SOUSA, Oncontrol Technologies, Portugal
PAULO SIMÕES, University of Coimbra, CISUC, Department of Informatics Engineering, Portugal

Industrial Automation and Control systems have matured into a stable infrastructure model that has been kept fundamentally unchanged, using discrete embedded systems (such as Programmable Logic Controllers) to implement the first line of sensorization, actuation, and process control and stations and servers providing monitoring, supervision, logging/database and data-sharing capabilities, among others. More recently, with the emergence of the Industry 4.0 paradigm and the need for more flexibility, there has been a steady trend towards virtualizing some of the automation station/server components, first by using virtual machines and, more recently, by using container technology. This trend is pushing for better support for real-time requirements on enabling virtualization technologies such as virtual machines and containers.

This article provides a systematic review on the use of container virtualization in real-time environments such as cyber-physical systems, assessing how existing and emerging technologies can fulfill the associated requirements. Starting by reviewing fundamental concepts related to container technology and real-time requirements, it goes on to present the methodology and results of a systematic study of 37 selected papers covering aspects related to the enforcement of real-time constrains within container hosts and the expected task latency on such environments, as well as an overview of container platforms and orchestration mechanisms for RT systems.

CCS Concepts: • **General and reference** → **Surveys and overviews** • **Computer systems organization** → **Embedded software**; **Real-time systems**; **Embedded and cyber-physical systems**;

Additional Key Words and Phrases: Real-time containers, industrial automation control systems, latency, virtualization of cyber-physical systems

This research work was co-financed by the iProMo (CENTRO-01-0247-FEDER-069730) and InGestAlgae (CENTRO-01-0247-FEDER-046983) projects, both co-funded by the European Regional Development Fund through Centro Regional Operational Program 2014/2020 (Centro2020) of the Portugal 2020 framework, as well as by the Smart5Grid project (POCI-01-0247-FEDER-047226), co-funded by FEDER via the Competitiveness and Internationalization Operational Program COMPETE 2020 of the Portugal 2020 framework.
Authors' addresses: R. Queiroz, T. Cruz, and P. Simões, University of Coimbra, CISUC, Department of Informatics Engineering, Coimbra, Portugal, 3030-290; e-mails: {rqueiroz, tjcruz, psimoes}@dei.uc.pt; J. Mendes, University of Coimbra, CEMMPRE, ISR, ARISE, Department of Mechanical Engineering, Coimbra, Portugal, 3030-290; e-mail: jermendes@isr.uc.pt; P. Sousa, Oncontrol Technologies, Coimbra, Portugal, 3000-108; e-mail: pedro.sousa@oncontrol-tech.com.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

This work is licensed under a Creative Commons Attribution International 4.0 License.
© 2023 Copyright held by the owner/author(s).
0360-0300/2023/10-ART59
https://doi.org/10.1145/3617591

ACM Reference format:
Rui Queiroz, Tiago Cruz, Jérôme Mendes, Pedro Sousa, and Paulo Simões. 2023. Container-based Virtualization for Real-time Industrial Systems—A Systematic Review. *ACM Comput. Surv.* 56, 3, Article 59 (October 2023), 38 pages.
https://doi.org/10.1145/3617591

1 INTRODUCTION

Industrial Control and Automation Systems (IACS) have been facing increasing demands, namely, since the emergence of the Industry 4.0 and Smart Factory concepts. The objectives pursued with this new paradigm shift imply a natural evolution of its cyber-physical systems, especially in terms of flexibility and scalability. These improvements add to the increasing relevance of cyber-physical systems, plus the increasing need for improved resilience and security. Coincidentally, in the IT domain there was an evolution with similar drivers that led to the progressive softwarization, virtualization, and consolidation of multiple systems.

After being adopted in the IT world, hypervisor-based virtualization technologies became a subject of interest for IACS applications, being adopted to consolidate less-demanding components with relaxed or no real-time requirements, such as historians and other **supervisory control and data acquisition (SCADA)** stations.

More recently, container-based virtualization matured enough to become a staple for modern IT infrastructures, due to its reduced resource overhead, often replacing or being used within hypervisor-based VM instances. Its unique characteristics may also be of value for IACS, due to the following reasons:

- It offers near-native performance, which is of utmost importance for real-time systems.
- Its isolation and resource control capabilities allow for the simultaneous deployment of multiple containers in the same host. This may be relevant when addressing mixed-criticality systems or when taking into consideration cost optimization.
- Its architectural concept leverages the development of modular software that simplifies the deployment, management, or reusage of software components, as encouraged by the IEC 61499 [43]. Also, multiple replica instances can be created as backup and dynamically deployed in case of failure.
- Being lightweight, containers are potentially more adequate for quicker instantiation and/or migration operations when compared with VMs. This is especially relevant for IACS applications, as the streamlined container overhead may help optimize latency as the result of reduced resource contention and/or elimination of intermediate abstraction layers (the latter is especially true for bare-metal deployments).

This article provides a systematic review of the use of container virtualization in real-time environments, namely, in cyber-physical systems such as IACS. Its goal is to assess how the technology is being used and if (and how) the current state of container-based virtualization technology can meet the requirements of real-time environments. This will provide the reader with a comprehensive understanding of the current state of research in this field, including the latest trends, approaches, achievements, and research gaps.

The remainder of the article is organized as follows: Section 2 overviews the topic of container virtualization, while Section 3 introduces the concept of real-time systems in the scope of industrial and automation control systems. Section 4 describes the methodology adopted in the systematic review process. The following five sections individually address each of the research questions that steered this review: Section 5 discusses how to ensure real-time constraints with the container host,

Section 6 discusses the latency that can be expected from container-based RT systems, Section 7 identifies which container platforms are mostly used for RT systems and why, Section 8 discusses container orchestration in RT systems, and Section 9 identifies the key open challenges in this domain. Section 10 concludes the article.

2 CONTAINER VIRTUALIZATION

Container virtualization has its roots in the creation of the chroot system call in the late 1970s. This may have been the first step towards process isolation by means of changing the root directory of a process and its children to a new location in the filesystem. In early 2000s, a new feature was introduced that allowed to subdivide a FreeBSD system into independent micro-systems and to assign different IP addresses to each micro-system. This partitioning mechanism became known as *FreeBSD Jails*, being later transposed to the Linux OS through a kernel patch.

In 2006, Google introduced "process containers," a feature designed to provide resource isolation, limitation of accounting capabilities. Later it would be renamed **"control groups" (cgroups)** and merged into the main Linux kernel tree, being combined with another kernel feature named "namespaces" to provide the building blocks for most Linux container implementations. In 2008, the first container system, named **Linux Containers (LXC)** [57] was announced, with the Docker framework [45] being introduced in 2013.

Since its inception up to this day, container technology has become a staple of modern computing environments, evolving from an abstraction of multiple Linux kernel features to sophisticated frameworks powered by tools specifically developed to support the execution, management, and orchestration of containers. However, there are other virtualization technologies whose origin is rooted in the 1970s, having evolved and survived up to this present day, and to whom containers may provide an alternative or even complementary role. These will be presented next.

2.1 Conventional Virtualization vs. Containers

In a strict sense, containers are the latest descendants in a line of system virtualization technologies that originated in the early days of mainframe computing and evolved up to this day. Nevertheless, containers are significantly different from other popular system virtualization technologies, as it is the case for classic hypervisors, something that this section intends to clarify, also discussing the benefits of containerization.

Currently there are three major types of virtualization technologies: type 1 hypervisor, type 2 hypervisor, and container-based (sometimes referred to as software-based virtualization or operating system-based virtualization), as illustrated by Figure 1.

Type 1 and type 2 hypervisor virtualization focus on the virtualization of the entire operating system, which each virtualized environment instance being designated as **virtual machine (VM)**. Type 1 hypervisors are commonly found in data centers, where they are used to accommodate different users or services on the same machine or cluster. Type 2 are mostly used in personal workstations to sandbox distinct working environments or to solve compatibility issues that require the use of different operating systems. As seen in Figure 1, type 2 hypervisors have an extra layer below the hypervisor, since these are installed on top of an OS, contrary to the type 1 hypervisors that are executed directly on top of the host hardware (bare-metal operation).

While hypervisors provide a complete hardware platform abstraction layer, container-based virtualization is supported by a thin layer provided by kernel-level mechanisms to host a wrapped package agglomerating code and all the dependencies for its execution. This means that a single operating system environment can host multiple containers, making it less resource-consuming than the previous alternatives [20, 34], albeit with less isolation than the one provided by VMs. Usually, but not necessarily, this approach follows the principle of one process per container, which

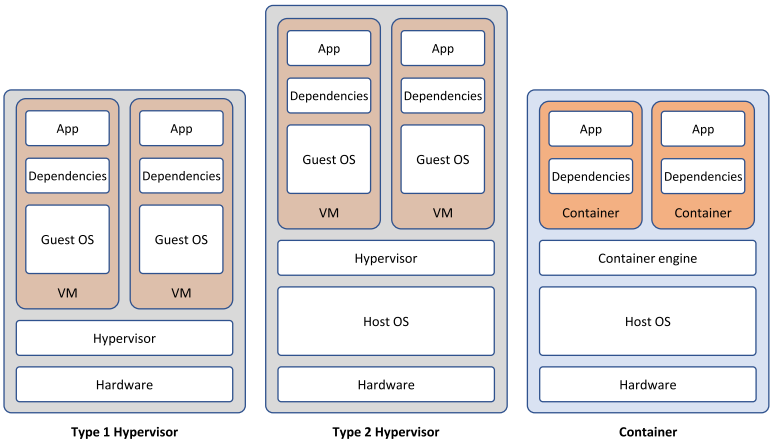

Fig. 1. Types of virtualization.

leverages the isolation between processes, improves the scalability, enables independent upgrades of different software/service components, facilitates its reusability, and simplifies the management of complex software/services. This lightweight virtualization is usually applied in micro-service environments, which are often based on modular distributed architectures. Two examples of such type of virtualization platforms are Docker [45] and LXC [57].

2.2 Anatomy of a Container Framework

A generic container virtualization framework, such as the one depicted in Figure 2 encompasses several components, which will be next presented and described in more detail:

- Container – a wrapper that includes all the necessary elements (environment variables, libraries, files, and other dependencies) for an application to be executed, allowing it to run in an isolated environment. It can assume two distinct states: running or not running. When running, it exists as a process being executed by the kernel. When not running, it exists as a container image. There are two types of containers: application containers, which package a single process or application; and system containers, which simulate a full operating system and tend to execute multiple processes simultaneously. Although containers were first conceived to be stateless, they can be stateful as long as there is some sort of persistent storage connected to the container.
- Container image – a file or bundle of files comprising everything that an application needs to be executed. It can simply consist of a binary file or it can have a more complex structure, including its own simplified operating system. It can be created from scratch or incrementally, on top of a pre-existing image (e.g., Alpine image) by adding the necessary elements or by re-using a pre-existing instance (which can be pulled from a public or private registry server). An image can have just a single layer, which means it is a base image with no parent layer, or it can have multiple layers, each one representing one or multiple changes that were made to its parent layer. This means that the parent image/layer always remains unchangeable. Considering the OCI specifications [46], the image exists as a tarball archive that contains a tarball file for each layer, plus JSON files with metadata necessary to run the container.
- Container host – the system where the container is running, i.e., where the container image is instantiated.

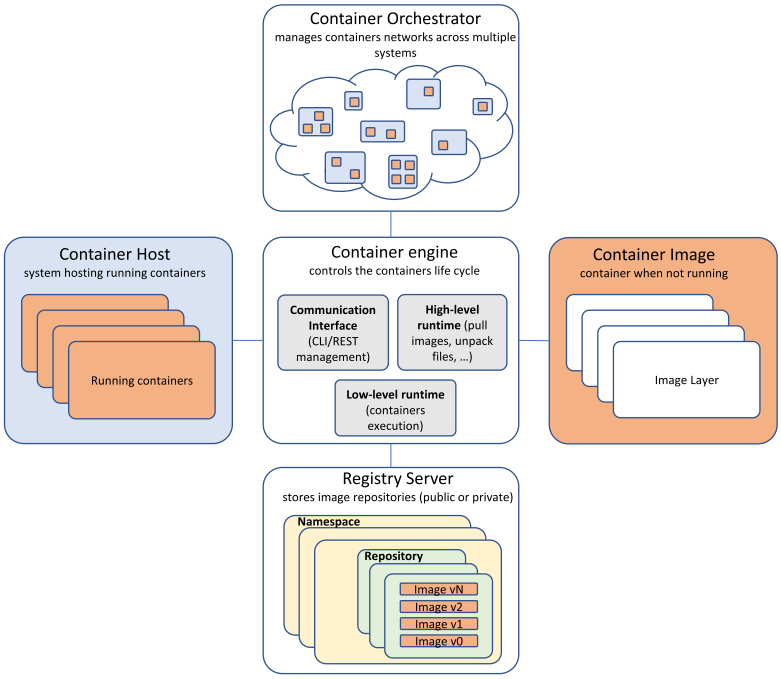

Fig. 2. An overview of the fundamental container platform building blocks.

- Registry server – a file server that stores container images. Container images are commonly organized by namespaces (not related to the kernel namespace) and, inside those, by repositories, each keeping a collection of different versions of the same image that may be distinguished using unique tags. By means of an access API, access control rules, and indexes, these file servers allow pushing and pulling container images.
- Container engine – the software that is installed on the container host and is responsible for controlling the life cycle of the containers. This makes it the central element in this ecosystem. It is usually composed of different tools that take on different tasks. This modularity is essential to leverage innovation through the possibility of changing each of these tools independently. Taking as an example one of the most used engines (Docker), we have (Figure 3): the *dockerd* daemon, which is responsible for handling the user input (commands send via CLI or REST API); the *containerd* daemon, which can be considered a high-level container runtime, being responsible for pulling and pushing container images from and to the container registry, managing the storage and network and supervising the running containers; and the *runc*, which is a low-level container runtime that uses the libcontainer library, being responsible for interacting with low-level Kernel features enabling the actual creation and running of containers.
- Container orchestrator – the software tool that automates much of the operational effort required to launch, schedule, and manage containerized systems in centralized or distributed environments. Especially useful when used in large-scale systems, this type of tool greatly empowers the use of containers by automatically dealing with everyday actions such as container scheduling, deployment, and networking, enabling horizontal scaling, load balancing, and self-healing, among others. Kubernetes is the most popular orchestrator and powers the containers management system of players such as Red Hat OpenShift [38],

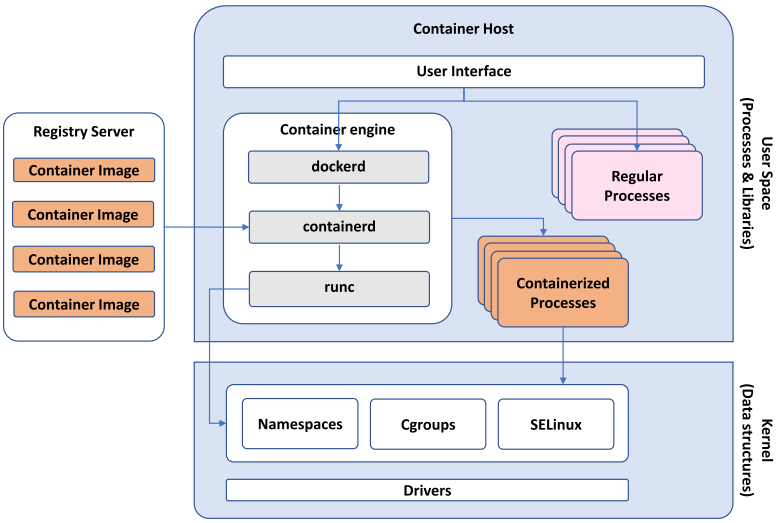

Fig. 3. Container execution (adapted from Reference [39]).

Google Kubernetes Engine [35], Amazon Elastic Kubernetes Service [3], and Microsoft Azure Container Instances [60].

Summing up, a container is a running instance of a container image that allows for code to be executed in an isolated way. It can be seen as a software package that bundles an application code and all its dependencies. When not being executed, it exists as a file or bundle of files that constitute the container image. When ordered to be executed, the container engine pulls the necessary meta-data and files from the registry server, unpacks the container image, and executes other required procedures (such as API calls to the Linux kernel) to run the container with isolation guarantees. At this point, the container is running on top of the Linux kernel and comes to exist as a Linux process.

The isolation is guaranteed by using Linux features such as: **kernel namespaces**, which act as an abstraction layer for global resources and enable the creation of an isolated workspace for each container (namely, by creating individual mounting points, network interfaces, user and process identifiers, etc.); **cgroups**, which allow the control and isolation of system resources by limiting the access to those (e.g., CPU, RAM, IOPS, network bandwidth); and **SELinux**, which ensures the isolation of resources through access control security policies that supervise the interaction between processes and system resources.

The structure needed for its existence in both states has been standardized by the **Open Container Initiative (OCI)** [29]. Created in 2015, the OCI is a combined effort of the main players in the container industry, with the aim of defining vendor-neutral, portable, and open specifications to leverage interoperability and innovation around this technology. One of the biggest contributors was Docker, which justifies the resemblance between Docker and OCI specifications and structures. OCI has already released three specifications [29]:

— Image Format Specification – defines the requirements for an OCI container image, including the image file on-disk format (at the top level, it is a simple TAR archive), the internal layout, and meta-data such as entry-point, hardware architecture, operating system, among others.

- Runtime Specification – defines how to run a container image compliant with the OCI image format specification. It includes specifications for the configuration and execution environment and others regarding the lifecycle of the container.
- Distribution Specification – defines an API protocol for the distribution of content. It can be used for pushing and pulling container images to or from the registry servers across platforms or any other type of content, since it was designed to be agnostic of content types.

In addition to OCI work, there are also other relevant specifications within the container universe, such as:

- **Container Runtime Interface (CRI)** [51] – a plugin interface developed by the Kubernetes team [28] that allows Kubernetes nodes to use multiple types of container runtimes. It defines the main communication protocol between the Kubelet ("primary node agent" that runs on each Kubernetes node) and the container runtime, thus, leveraging the ecosystem compatibility and allowing for the execution of the preferred runtime on each container node.
- **Container Network Interface (CNI)** [27] – a Cloud Native Computing Foundation [26] project that specifies and includes coding libraries for writing plugins to configure Linux container network interfaces. Moreover, it also focuses on the garbage collection of resources once containers are deleted.
- **Container Network Model (CNM)** [18] – another specification for configuring network interfaces for Linux containers. It was proposed by Docker and, like CNI, it empowers container virtualization by enabling multiple core networking functionalities and configurations.

The growing adoption of container virtualization (especially in the IT world), the emergence of standards that allow the normalization of the entire ecosystem, and its lightweight virtualization capabilities with isolation guarantees, make it a technology with the potential to be widely adopted as well by the OT domain, namely, for real-time systems. Thus, and complementary to the topic of container technology, the next section will introduce real-time systems, also presenting the cornerstone concepts supporting them.

3 REAL-TIME SYSTEMS

By definition, a real-time system is a system that can respond to an event within pre-specified and guaranteed timing constraints. This time limit is called a deadline and often comprises a few milliseconds or even microseconds. It is expected for such a system to be able to receive data from the surrounding environment, process it, and, if needed, trigger some mechanism that influences the environment at that point in time—this capability to respond correctly and in a timely manner is closely related to the system's determinism and predictability. By definition, one knows that a deterministic system involves no randomness, which in this case means it should always produce results within the same time frame. Often, these two concepts are merged and simply referred to as determinism.

Real-time systems can be divided into three distinct categories according to the impact of failing to respond within the pre-defined deadline (Figure 4):

- **Hard real-time (HRT)** – The inability to meet a deadline results in a system failure. Responses following the deadline are automatically devoid of value.
- **Firm real-time (FRT)** – Deadlines can be infrequently missed without causing a system failure, however, there may be a degradation in the quality of service. Responses following the deadline are automatically devoid of value.

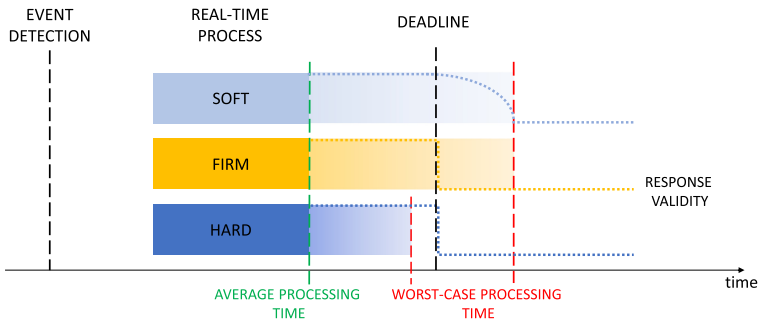

Fig. 4. Real-time systems categories (from [67]).

Table 1. Industrial Systems End-to-end Latency

Service	End-to-end latency	Jitter
Factory automation (motion control)	1 ms	1 μs
Factory automation	10 ms	100 μs
Process automation (remote control)	50 ms	20 μs
Process automation (monitoring)	50 ms	20 ms
Electricity distribution (medium voltage)	25 ms	10 ms
Electricity distribution (high voltage)	5 ms	1 ms
Intelligent transport systems (infrastructure backhaul)	10 ms	2 ms
Remote Control	5 ms	1 ms

—**Soft real-time (SRT)** – Deadlines can be infrequently missed without causing a system failure. Responses following the deadline are still considered; even so, there may be a degradation in the quality of service.

The time set for the deadlines depends on the system and context in question, and it is the designer or programmer's responsibility to set it accordingly. There is no "one size fits all" value. However, the literature can give us some guidance. For example, Khan et al. [48], while examining the application of cloud solutions in the Industry 4.0, present some data regarding the QoS in such industrial environments. The latency for motion control is placed between 250 μs and 1 ms, for augmented reality in 10 ms and for conditioning monitoring in 100 ms. Greifeneder et al. [37] studied the overall response time in industrial networked automation systems taking into consideration distinct architectures with different control systems, namely, event-driven systems using interrupts and time-driven using clock cycles. Multiple case studies were considered, and the average response time between an input change, the execution of the internal processing algorithms, and the subsequent output activation were comprised between 18 ms and 37 ms. There are also indicative values for the end-to-end latency [23]—as shown in Table 1—from which the deadlines can be gauged.

Next, we discuss the two main properties relevant for deterministic systems: latency and jitter.

3.1 Latency

Conventional computing is oriented towards throughput optimization, which means that coalescing or deferred processing mechanisms are often implemented to optimize resource usage at the cost of reduced predictability or increased execution delays for certain tasks. In fact, when it comes

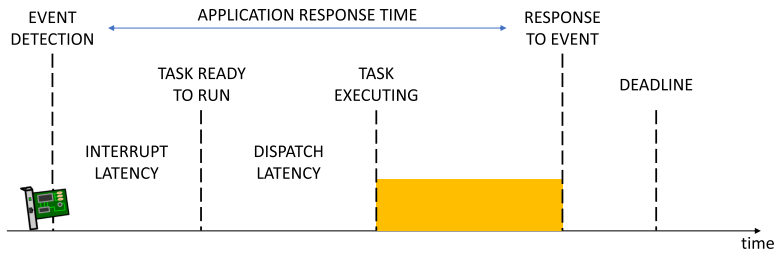

Fig. 5. Processing latency (from [67]).

to latency, a real-time system must be concerned with the delays for end-to-end communications and host-level processing levels, whose sum defines the maximum tolerance allowed by the system. Moreover, each of these components is also affected by other factors.

For communications, latency is the sum of multiple delays that may arise from different sources—for instance, a packet switched network has a series of inherent delay sources that must be accounted for, namely:

- Propagation delay – the time required for a packet to travel from the sender to the receiver. It can be computed considering a function of distance over the speed the signal propagates along the communication channel. In reasonably controlled environments, this value is usually uniform for each communication channel type.
- Transmission delay – the time required to push an entire packet into the communication channel. Its value can be computed considering a function of the packet's length over the transmission rate. Normally, the specifications of the hardware being used define the transmission rate.
- Processing delay (not to be confused with the processing latency "subgroup") – the time required for a node to process a packet and be able to check for errors and determine its next destination.
- Queuing delay – the time that a packet spends in a queue waiting to be processed or transmitted.

When the packet delivery takes place, the network device rises an interrupt so the system becomes aware of this event. The real-time process (usually referred to as RT task) responsible for dealing with this event has first to be scheduled, then processed, and finally a response to the event is achieved. Thus, processing delay occurs after the packet is delivered to a real-time system control device. It is possible to identify different sources of delay in the processing latency (Figure 5), among which two stand out:

- Interrupt latency – When the interrupt is raised, the system may not be available to handle this interrupt at this exact time due to circumstances that may be locking out interrupts. Also, actions like having the processor save the state of execution, and the interrupt processing itself, add extra delay to the process.
- Dispatch latency – After the interrupt is handled, the RT task becomes ready to run and is scheduled for processing according to the scheduling policies. This dispatch process generates delays caused by context switching, scheduling, and dispatching, among other conflicts that may arise in the process.

Both the interrupt and the dispatch latency depend on the operating system's main purposes that, in turn, define how its kernel is programmed. For example, if the OS goal is to prioritize

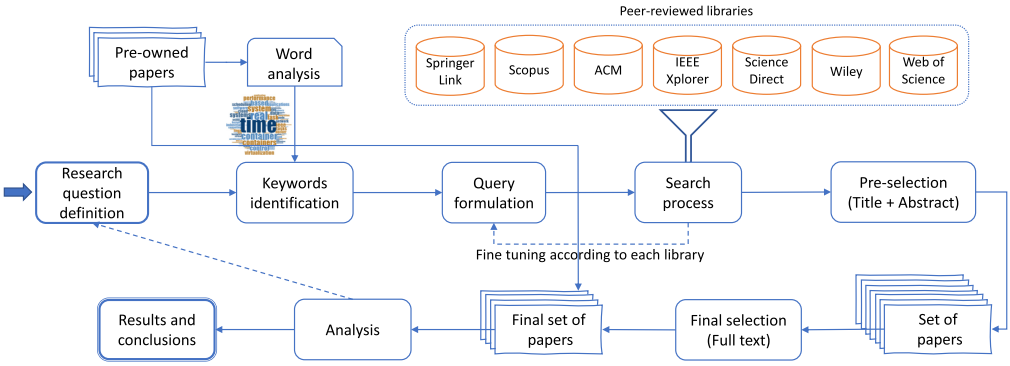

Fig. 6. Systematic search process roadmap.

throughput, then the kernel scheduler will probably apply a non-preemptive policy that will increase the dispatch latency; if the OS wants to deal with mutual exclusion problems and to assure that only one process is executed in a critical region at a time, then it may increase the maximum time that interrupts can be disabled, nevertheless, the interrupt latency will increase.

The aforementioned aspects, which are relevant for bare-metal OS deployments, also provide an intuition about the potential penalty induced by the introduction of intermediate abstraction layers, as it is the case for hypervisors, whose inherent overhead might vary significantly due to factors such as the hypervisor type (type 1 or bare-metal vs. type 2 or OS-hosted) or the specific virtualization techniques being used, such as full virtualization, paravirtualization, or hw-assisted virtualization [67]. Accordingly with the specific techniques being used, hypervisors may need to handle aspects such as virtualized device drivers, interrupt steering mechanisms, or nested memory management (just to mention some examples), which have an implicit penalty on host processing times when services are hosted within virtual machines. From this perspective, containers offer the advantage of providing minimal overhead for the execution environment (close the bare-metal OS deployments), offering near-native performance due to reduced resource contention and/or elimination of intermediate abstraction layers.

3.2 Jitter

Since predictability implies producing results always within the same time frame, it is essential to ensure that the latency variation—also referred to as jitter—is as small as possible and within the RT task restrictions. The lower the jitter, the higher the predictability of the system.

In real-time systems, having a normalized latency is at least as important as having low latency. As presented in Table 1, the jitter value is always more restricted than the latency value. A response being achieved earlier than expected can result in synchronization problems, but a response being achieved later than expected can result in its invalidation.

4 SYSTEMATIC REVIEW PROCESS

As already mentioned, the systematic review presented in this article focuses on the use of Containers (a broad-scope technology) to the specific domain of real-time systems.

Based on the guidelines for performing systematic literature reviews as presented by Kitchenham et al. [49] and other authors [24, 58], the review process defined for this research work entailed three main phases: (A) planning, (B) implementing, and (C) reporting the review. Figure 6 presents a roadmap that outlines the practical application of these principles to the study hereby presented.

Table 2. Research Questions

#	Research Question	Goal
RQ1	How can real-time be ensured within the container host?	To identify which techniques can be used to ensure that real-time task requirements are met.
RQ2	What is the expected order of magnitude for the real-time task latency when using containers?	To estimate the guaranteed latency that can be achieved using container-based RT systems to assess their compliance with the requirements of IACS.
RQ3	Which container platforms are better suited to real-time systems?	To identify which container platforms are more used in real-time systems and why.
RQ4	Is container orchestration being used in the real-time systems scope?	To understand if and how the orchestration of containers is used to leverage this technology in the scope of RT systems.
RQ5	What are the key open challenges of applying container technology to real-time systems?	To identify the key remaining challenges towards the adoption of container-based RT systems for IACS.

4.1 Planning of the Review Process

This first phase corresponds to the definition of the rationale and goals for conducting the review, as well as to identify the relevant research questions and the basic review procedures to be adopted. More specifically, this survey aims to assess the applicability of container virtualization technology to real-time environments, addressing the research questions presented in Table 2.

To identify the best-matching keywords to use in the search for relevant or related works, we used the specific methodology. First, we performed a lexical analysis of a set of papers identified in previous research that somehow addressed this topic to select the most frequently used words—these are shown in Table 3, which lists the 22 topmost used words for title, abstract, and text. These three categories are also used to classify each word accordingly to the ones in which it appears in, as well as the respective number of occurrences (e.g., the word "time" appears in all categories, thus being classified as III, with a per-category frequency of 16 times for *titles*, 79 for *abstracts*, and 2,852 times for *full text*).

Next, the following well-known online digital libraries were used: ACM [25], IEEE Xplore [44], Science Direct [21], SpringerLink [63] Scopus [22], Wiley [74], and Web of Science [14]—the latter aggregates information from multiple sources—thus, enabling access to papers related to the research. By resorting to these libraries, composed of peer-reviewed studies published in well-reputed venues, it was possible to indirectly (partially) ensure the scientific quality of the papers referenced in this research.

The final set of papers in this review was selected taking into consideration the criteria presented in Table 4. Only papers published between 2016 and 2022 were considered due to the strong evolution of container technology in recent years and since the focus was on the current state-of-the-art.

Regarding query formulation, some tests were carried out to understand the best way to build the search query. For this matter, the IEEE Xplore digital library was used as reference. After a few attempts, the keywords that seemed to produce the best results were selected, as well as the combination of the different elements where to search for, leading to the query shown in Listing 1. Table 5 shows the results obtained for the conjugation of the different elements considered when using that search query. It was concluded that the search by title, as well as by abstract, returns

Table 3. Lexical Analysis

Title			Abstract			Full text		
time	III	16	time	III	79	time	III	2,852
real	III	15	real	III	69	real	III	1,590
industrial	III	14	virtualization	III	55	container	III	1,362
container	III	13	based	III	45	system	III	1,212
based	III	12	container	III	39	containers	III	1171
virtualization	III	12	containers	III	35	based	III	1,091
containers	III	10	systems	III	35	control	III	1,036
control	III	9	automation	II	31	task	I	994
applications	III	6	industrial	III	30	systems	III	981
systems	III	6	virtual	I	27	performance	III	950
automation	II	5	control	III	25	virtualization	III	885
software	III	5	paper	I	25	ieee	I	876
architecture	II	4	architecture	II	23	data	I	791
embedded	I	4	software	III	22	tasks	I	783
scheduling	III	4	technology	I	22	cloud	II	719
criticality	I	3	applications	III	21	applications	III	706
distributed	I	3	cloud	I	19	software	III	700
lightweight	I	3	performance	III	19	scheduling	III	692
linux	II	3	scheduling	III	18	network	I	658
mixed	I	3	solution	I	18	docker	I	654
performance	III	3	requirements	I	16	industrial	III	629
survey	I	3	computing	I	14	linux	II	621

Table 4. Study Selection Criteria

Inclusion criteria	
(1)	In scope with the subject (real-time containers virtualization)
(2)	Addressing the research questions
(3)	Providing proper argumentation and/or validation
(4)	Written in English
(5)	Primary study
(6)	Published between 2016 and 2022
(7)	Accessible
Exclusion criteria	
(1)	Failing to satisfy the inclusion criteria

a considerably smaller number of results when compared with searching in the paper's full text. Combining any of the first elements with the full text produced a slightly more refined result. As such, those three elements were combined as a way to achieve better results.

4.2 Implementation of the Systematic Review

In this phase, we adopted an unbiased search strategy to identify as many studies related to the research questions as possible, to select those that were within the inclusion criteria, and to extract and synthesize relevant data.

Table 5. Search Query Statistics

Search Elements	Year Range	
	2000–2022	2016–2022
Title	270	160
Abstract	1,285	823
Full Text	2,868,122	1,218,381
Title & Abstract	51	39
Title & Full Text	241	144
Abstract & Full Text	1,070	706
Title & Abstract & Full Text	43	35

Listing 1. Adopted Query

```
((“Document Title”:real OR “Document Title”:time OR “Document Title”:industrial OR
    “Document Title”:automation OR “Document Title”: embedded) AND (“Document Title”:
    container* OR “Document Title”:virtualization))
AND
((“Abstract”:real OR “Abstract”:time) AND (“Abstract”:virtualization OR “Abstract”:
    container*) AND (“Abstract”:industrial OR “Abstract”:automation OR “Abstract”:control
    ))
AND
((“Full Text Only”:real time AND “Full Text Only”:container* AND “Full Text Only”:
    virtualization) AND (“Full Text Only”:industrial OR “Full Text Only”:control) OR
    (“Full Text Only”:latency OR “Full Text Only”:performance OR “Full Text Only”:
    scheduling))
```

To identify relevant research work, the search query was applied to the aforementioned digital libraries: ACM, IEEE Xplore, Science Direct, SpringerLink, Scopus, Wiley, and Web of Science. Some fine-tuning had to be made during the search process, since the search commands and/or options slightly change across the different libraries.

Despite the efforts for producing the most efficient search query possible, it was necessary to carry out a pre-selection of the papers to download after the search in each of the libraries. This filtering was done based on the title and abstract of each paper. After eliminating some papers that were obviously out of scope, the selected papers were retrieved.

Finally, retrieved papers were subject to a filtering process based on a full text analysis, with several being discarded due to their shallow technical depth and/or lack of relevant information. The final count amounted to a total of 37 selected papers. Table 6 identifies where the selected works were published, the type of publication, the publisher and/or entity responsible for the event, and the reference for the papers.

The papers constituting the final selection were analyzed in detail, having in mind the aforementioned research questions. The results and conclusions taken from this analysis effort are reported in the next five sections, organized according to the aforementioned research questions.

5 HOW TO ENSURE REAL-TIME WITHIN THE CONTAINER HOST?

Standard container virtualization was not designed with real-time environments in mind and does not natively support real-time requirements. When installed on a general-purpose operating system, the tasks being executed inside containers will eventually be processed under a best-effort policy (as they are subject to kernel scheduler policies), thus not complying with real-time requirements. As such, containerization needs to be complemented with other techniques, so it can deliver

Table 6. Publications

Conference/Journal Name	Type	Publisher	# of papers	Reference
Design, Automation & Test in Europe Conference & Exhibition	Conference	IEEE	1	Chen et al. [10]
Embedded Operating Systems Workshop	Workshop	ACM SIGBED	1	Abeni et al. [1]
Euromicro Conf. on Real-Time Systems	Conference	Euromicro	2	Barletta et al. [5] Cinque et al. [11]
Euromicro Conf. on Software Engineering and Advanced Applications	Conference	Euromicro	1	Goldschmidt and Hauck-Stattelmann [33]
IEEE Access	Journal	IEEE	2	Morabito [62] Okwuibe et al. [64]
Int. Conf. on Advanced Information Networking and Applications	Conference	Springer	1	Li et al. [53]
Int. Conf. on Automation Science and Engineering	Conference	IEEE	1	Telschig et al. [77]
Int. Conf. on Cloud Computing	Conference	IEEE	1	Cucinotta et al. [15]
Int. Conf. on Cloud Computing Technology and Science	Conference	IEEE	1	Hofer et al. [42]
Int. Conf. on Cloud Networking	Conference	IEEE	1	Carvalho et al. [8]
Int. Conf. on Dependable Systems and Networks Workshops	Conference	IEEE/IFIP	1	Cinque and Cotroneo [12]
Int. Conf. on Edge Computing	Conference	IEEE	1	Cucinotta et al. [16]
Int. Conf. on Emerging Technologies and Factory Automation	Conference	IEEE	5	Garcia et al. [32] Albanese et al. [2] Govindaraj and Artemenko [36] Struhàr et al. [75] Krüger et al. [50]
Int. Conf. on Engineering, Technology and Innovation	Conference	IEEE	1	Tasci et al. [76]
Int. Conf. on Industrial Informatics	Conference	IEEE	1	Sollfrank et al. [73]
Int. Conf. on Intelligent Transportation Systems	Conference	IEEE	1	Masek et al. [59]
Int. Conf. on Mechatronics and Machine Vision in Practice	Conference	IEEE	1	Hinze et al. [40]
Int. Symposium on Software Reliability Engineering	Symposium	IEEE	1	De Simone and Mazzeo [17]
Internet of Things Journal	Journal	IEEE	1	Kaur et al. [47]
Journal of Cleaner Production	Journal	Elsevier	1	Lin et al. [54]
Journal of Systems Architecture	Journal	Elsevier	1	Goldschmidt et al. [34]
Open Journal of the Industrial Electronics Society	Journal	IEEE	1	Liu et al. [56]
Real-Time Systems Symposium	Symposium	IEEE	1	Cinque and De Tommasi [13]
Recent Advances in Computer Science and Communications	Journal	Bentham Science	1	Yadav et al. [79]
Sensors	Journal	MDPI	1	Lee et al. [52]
Symposium on Applied Computing	Symposium	ACM/SIGAPP	1	Moga et al. [61]
Symposium on Security and Privacy	Symposium	IEEE	1	Cervini et al. [9]
Systems Engineering	Journal	Wiley	1	Hofer et al. [41]
Transactions on Industrial Informatics	Journal	IEEE	3	Yin et al. [80] Singh et al. [71] Sollfrank et al. [72]

real-time performance. Our systematic review identified several strategies to achieve this, including methods based on standalone/single kernel, methods based on RT co-kernels, scheduling-based methods, and other approaches, as discussed next.

5.1 Methods Based on Standalone/Single Kernel

This category entails all the methods that tweak the kernel so it becomes preemptable according to the priority of each thread, such as the PREEMPT_RT Linux kernel patch [30] or Real-Time Linux [31] that incorporate the PREEMPT_RT patch functionalities into the Linux kernel mainline—not to be confused with RTLinux. Such preemption capabilities allow software developers to easily create (user-space) real-time applications [68], enabling tasks with higher priority to be given precedence over other tasks that may be executing at the time. In some cases (depending on the preemption level), it may even be possible to preempt critical sections like interrupt handlers. An interrupt handler can be treated as a "normal" thread and its priority modified. Such an approach allows a real-time task running at the user space to have priority even over interrupts, thus guaranteeing a more deterministic behavior. Next, we review the more relevant works based on this method.

Masek et al. [59] evaluated sandboxed software deployments for real-time systems, namely, self-driving heavy vehicles. The authors investigated to what extent the execution environment influenced scheduling precision and input/output performance of a given application using a real-world application from a self-driving truck for evaluation purposes. Four distinct execution environments were used: (i) a conventional Linux kernel, (ii) a native real-time Linux (preempt_rt) kernel, (iii) a Docker container with a conventional Linux kernel, and (iv) a Docker container with a real-time Linux (preempt_rt) kernel. It was concluded that using Docker containers had a negligible impact on the performance of the system and that, on average, the specified real-time deadlines were not violated. Also, it was concluded that choosing the correct kernel was of the utmost importance, since swapping kernels translated into a significant variance in terms of scheduling precision and input/output performance. The obtained results were considered in line with referenced research that concluded that the processing latency of a task running inside a Docker container could be improved by 13.9 times (from 446 μs to 32 μs) when using a real-time Linux kernel, compared to using general-purpose kernels. The system load was another factor pointed out by the authors, as the controlled experiment with a more demanding load placed the highest standard deviation of around 7,000 μs, versus the uncontrolled one that was less exhausting and had a higher standard deviation (around 60 μs).

Moga et al. [61] investigated the use of operating system virtualization to support industrial automation systems, such as motor drive control applications with cycle times between 1 ms and 250 μs. With this aim in mind, authors analyzed how industrial automation platforms could be consolidated by using Docker containers and also performed an experimental study about container-induced overhead. As a way of guaranteeing real-time performance, the evaluation was undertaken using a container host running a Linux Ubuntu distribution with the real-time kernel patch 3.14.12-rt9, with all the containers being run with the *–privileged=true* flag so they could have privileged access to the OS functions and resources. The evaluation took into consideration two distinct aspects: (i) timing accuracy, which addresses the processing latency during the cyclic behavior of a control application; and (ii) virtual networking performance, which relates with the application overhead imposed by inter-container virtual networking. It was concluded that container technology could meet real-time system requirements, offering near-native performance with the added benefits of increased modularity, flexibility, and portability to real-time systems. However, some concerns were raised related to the optimal allocation of containers to resources, especially when multiple containers running real-time applications need to access shared resources.

Goldschmidt et al. [33, 34] presented a container-based architecture for a multi-purpose controller as a replacement for the typical programmable logic controllers and other automation controllers with working cycles between 100 ms and 1 s. This approach intended to create more flexible functions deployment to support innovation and to address problems related to the support of legacy systems in which the control code is often tightly bonded with specific hardware. Using a containerized system, it becomes possible to support multiple execution engines running over the same hardware at the same time, as well as to emulate legacy systems. In addition to using the real-time kernel patch (PREEMPT-RT) as a way to guarantee real-time performance, the authors mention the need to use the *cap-add=sys_nice* flag (assuming the Docker platform) as a way to allow tasks being executed inside the containers to access real-time priority within the host system. Multiple tests under different scenarios were executed: first, using a Docker container running a simple application and, later, running a QEMU PowerPC emulator in an LXC container. According to the provided results, the containerized execution of control applications suffers a negligible and constant overhead, thus meeting real-time requirements even when emulating legacy hardware—thus confirming the possibility of replacing legacy hardware components without having to change the control execution environment (e.g., migrating the unmodified binary legacy software components to the new hardware). Those results also hint at the possibility of achieving highly automated flexible function deployment by using the container registry and components deployed through the exposed APIs.

Albanese et al. [2] also relied on the single kernel approach and used Docker with a Linux-based operating system with the preempt_rt patch applied. To increase the determinism of the system, multiple BIOS functionalities were deactivated (e.g., power-saving modes, frequency scaling, CPU hyperthreading, and Sub-NUMA clustering). Different cores were explicitly assigned for the kernel and OS jobs (1–2) and for the real-time application instances (2–7), and each core used private L1/L2 caches and shared the L3 cache. This way, each instance was able to run exclusively in its allocated core(s) with extra isolation guarantees. Unlike most research works, the authors focus on the network performance and its ability to cope with real-time requirements in containerized virtual environments. Four different network technologies were evaluated: three Docker-supported software solutions (Host, Bridge, and MACvlan), and the **Root I/O Virtualization (SR-IOV)** hardware-assisted solution. All of these were submitted to multiple tests, which consisted of a real-time task with a 2 ms deadline for receiving packets transmitted from an external packet generator. Tests with and without a concurrent workload were executed. Network latency and missed packets were measured and registered. The median values ranged between 189 μs (SR-IOV) and 599 μs (MACvlan), being that the first presented very stable results along all the distinct tests. No missed packets were registered except for the MACvlan—the authors eventually realized that this may be related to the way the MACvlan Linux driver deals with multicast transmissions, since after switching to unicast transmissions the results improved dramatically. It was concluded that, albeit software-based solutions like OS-level Bridges and MACvlan support are more sensitive to network and system load fluctuation, this can be mitigated by using VLANs with the former and unicast traffic with the latter, thus improving compliance with latency-sensitive applications. SR-IOV hardware-assisted virtualization was shown to be more robust and less prone to load variations while enabling NIC sharing in a straightforward way, which makes it a good choice for containerized virtual environments.

Sollfrank et al. [73] envision a trend where the automation pyramid is shifting from a static form to a dynamic state, with the virtualization of applications becoming more present in **Cyber-Physical Production Systems (CPPS)**. Acknowledging that such systems are often connected through a network and have a distributed control logic (thus being referred to as **Distributed Networked Control Systems (DNCS)**), the authors note the importance of studying not only

the nodes' processing latency but also the propagation delay. Being so, the usability of containers based on Docker for industrial time-sensitive applications was analyzed taking into consideration those two factors. Three nodes interconnected through an Ethernet switch were used, two of which exchanged UDP packets representing messages being used by real-time tasks. For guaranteeing a deterministic processing behavior on the nodes, a real-time Linux Kernel was used (4.4.5-rt17-v7+ SMP PREEMPT). Also, the *–cap-add=sys_nice* and the *–ulimit='rtprio=98'* flags were used together to assign the real-time task with the second highest priority (being the highest priority assigned to the bash script responsible for starting the UDP-communication application inside the containers). It was observed that the processing delay on the nodes did not exceed 0.5 ms, which is below the threshold defined for the rt-task, thus confirming the system as real-time capable. The mean on-node processing delay when using containers was 117 μs, and 74 μs when not using. Regarding the network propagation delay, a small difference was identified. With Docker, the mean delay was 513 μs, and without Docker it was 508 μs. However, this difference was in the range of the NTP clock synchronization where some outliers were up to 700 μs. The presented conclusions state that the containerized environment is suitable for real-time applications if the priorities are well assigned. Also, network delay has to be considered, since the standard Ethernet is not deterministic and has some time delay outliers that may go over the applications' processing time. Later, in Reference [72], the same team addressed a relevant case study by considering a multi-thread application. By observing the latency values of a dual-thread application when executed in an isolated Docker container, versus running in a non-isolated non-containerized environment, they also concluded that multitask applications benefited from the isolation offered by the container environment, since it achieved a more deterministic behavior.

Hinze et al. [40] also experimented with a single kernel method, using a Linux Ubuntu distribution with the PREEMPT_RT kernel patch. In this case, the goal was to create an environment capable of being executed on local or cloud environments that could simulate the handling of deformable materials by industrial robots, such as cables or ropes, with real-time constraints. This is more of a challenge if considering that the handling of this type of material is characterized by nonlinear, location, and time-dependent equations for the objects' behavior—thus, a highly accurate and detailed simulation environment, capable to run in a deterministic time frame, is needed to obtain reliable results. Also, the authors looked to fulfill other non-functional requirements such as maintainability, reusability, flexibility, and portability. For this matter, the Docker container platform (runtime and orchestration) was used together with a RT-capable interface, implemented using ZeroMQ. Some tests were performed by means of a face-to-face comparison of a non-virtualized system versus a containerized version, under the same environmental conditions (with and without concurrent load). Although the non-virtualized presented slightly better results in terms of mean latency (3.99 μs vs. 4.07 μs) and standard deviation (0.0748 μs vs. 1.4956 μs), when no concurrent workload was present, the containerized environment performed considerably better when in presence of parallel workload (mean latency 4.82 μs vs. 4.12 μs; standard deviation 58.0151 μs vs. 0.4102 μs). Such results hint at determinism improvements when using containers in shared environments. Nevertheless, the authors mention that further studies must be made to further demonstrate the real-time capabilities of the system. It is also concluded that the non-functional requirements mentioned above were achieved in part due to the isolation and encapsulation offered by the usage of containers.

In Reference [41], Hofer et al. proposed a flexible architecture for real-time control systems capable of exploiting container virtualization and leveraging off-the-shelf technologies in cloud environments. The architecture is divided into three distinct layers. The first is the monitoring and management of the cloud infrastructure and services. Decisions are taken based on the analysis of the aggregation of data collected globally. The second layer, named "Control Cluster," is

composed of several nodes and is where all the process control and related services are taken care of. All the services (real-time or best effort) that interact with on-premises devices are located in this layer. This layer also hosts the orchestration software that is responsible for increasing resource utilization without significantly impacting the system determinism. As such, the advantages and disadvantages of using static and dynamic resource scheduling strategies are studied, using a dynamic orchestrator that uses probabilities based on runtimes, contemporaneity factors, and the probability of exceeding a given worst-case execution time, to allocate resources. Based on several tests performed in cloud environments, the authors conclude that the conjunction of the Ubuntu 16.04 LTS operating system with the PREEMPT_RT real-time patch, and the Docker container platform, provide the best latency among the low-maintenance solutions analyzed. Also, tasks with longer periods suffer less from external noise and tasks with shorter periods (e.g., 1 ms control loop for motion control systems) may need a specific scheduler, since the commercial schedulers' refresh rate limit may not be able to accommodate such requirements. Those tests also hint at the possibility of exceeding over 90% of CPU resources without affecting the determinism of the system (depending on the tasks' properties). Finally, it also concluded that generic shared cloud resources may viably host less-critical operations with periods around 100 ms (being that the worst observed runtime variation was around 126 μs).

The centralized protection and control concept for electric substations, presented in Reference [8] (which documents the core contribution of an MSc. thesis [7], later published in 2023), constitutes one of the most realistic efforts towards controller equipment virtualization, covering the issues surrounding the consolidation of mixed-criticality workloads (hard-RT and general-purpose) to implement virtual **Intelligent Electronic Devices (vIEDS)** in commodity x86 hardware, with support for network determinism. For this purpose, the authors adopted a VM-hosted containerized architecture that resorts to the PREEMPT_RT patch together with HugePage support (1 GB) to increase **Translation Lookaside Buffer (TLB)** hits and lock pages in memory, disabling hyperthreading, CPU C-states, power-management features, and dynamic frequency scaling, with the kernel option for omitting scheduling-clock ticks on CPUs that are either idle or that have only one runnable task (CONFIG_NO_HZ_FULL=y), and using both core affinity and partitioning. Obtained results achieved an average latency of 14 μs in single instance tests (with a maximum of 20 μs) and a maximum latency below 450 μs under stress (with 20 coexisting instances) using the cyclictest tool. In a scenario with 20 coexisting instances, authors verified that vIEDs running on real-time VMs were able to process IEC 61850 Sampled Value messages without violating the time constraints imposed by the standard, with a maximum latency value below 450 μs and average values under 100 μs.

From the reviewed studies, it can be concluded that standalone/single kernel methods seem to support real-time behavior, complying with latency requirements in the microsecond range. However, considering the diversity of the studies, targeted architectures, and the different approaches adopted in each, it is difficult to provide a more precise indication of the guaranteed latency ranges. The use of the *–cap-add* and *–ulimit* flags while using the Docker platform should be highlighted for their relevance in guaranteeing containerized task compliance with real-time requirements.

5.2 Methods Based on Real-time Co-kernels

A **general-purpose Linux kernel (GPLK)** allows non-preemptible routines to be executed. This can be problematic when a real-time task is ready to be executed but is kept waiting until the non-preemptible routine processing finishes. In methods based on RT co-kernels, a second kernel that runs aside from the main kernel and has higher priority, executes tasks that are time critical, such as real-time tasks and interrupt management [68]. In such circumstances, the GPLK is treated as a thread with idle priority, which means it has the lowest priority and is only given the chance

to run and execute its own scheduler when the co-kernel becomes idle after executing rt-tasks to completion. If in the meanwhile an interrupt is triggered, then it will be held until the rt-task being executed finishes, using a deferred processing strategy. If there is a real-time handler associated with that interrupt, then it will be executed by the co-kernel—otherwise, it is handed to the GPLK. One of the major differences of this method, in terms of software development, is the need to use specific API/system calls according to the co-kernel approach. However, in some cases, the existence of such specific API provides the application programmer with more refined control over the execution of its real-time application, possibly making applications more deterministic and predictable. The most frequent open-source co-kernel distributions are Xenomai Cobalt [78] and RTAI [69]. Next, we review the more relevant works based on this method.

Cinque et al. [11–13] applied the concept of real-time containers to mixed-criticality systems to take advantage of many-core architectures while trying to guarantee naming, temporal, and fault isolation of tasks. The proposed architecture relied on Docker containers on top of a Linux operating system patched with RTAI. The containers were marked with different criticality levels to define their relative importance—a kind of global fixed-priority assignment. Also, a specific library (rt-lib) was implemented to provide a transparent mapping of rt-tasks on the underlying real-time core, according to the criticality level of the containers and to expose standard primitives to rt-tasks inside the containers. Some modifications to other libraries were also necessary, to achieve naming and temporal isolation. Re-mapping the priority of rt-tasks was also done to assure that rt-tasks running into low-criticality containers could not preempt rt-tasks running in higher-criticality containers. However, the authors conclude that fault isolation was mainly guaranteed by the underlying technology and that it would need refactoring to increase its robustness.

Barletta et al. [5] rely on Xenomai and Docker to create a platform for mixed-criticality systems where real-time containers host real-time tasks and run on top of an operating system with real-time scheduling capabilities, side-by-side with non-real containers running non-real-time tasks. Among others, the proposed solution presents a hierarchical scheduling (SCHED_DS) based on the grouping of real-time tasks, "resource's budgets" for each group, and two levels of run queues—that suppress the need for active monitoring the tasks already running within containers but proactively isolate the CPU. The newly introduced Xenomai SCHED_QUOTA policy is used to impose CPU quota intervals to threads. This policy was customized to achieve a truly hierarchical scheduler able to overcome some limitations identified by the authors. Authors also present a feasibility checker that verifies in advance (in terms of network and CPU) if a new RT container can be created without negatively affecting already-running containers. To achieve network bandwidth guarantees, the proposed solution was integrated with the RTnet [70] real-time networking stack. RTnet wrappers are used within containers to isolate from disturbances coming from other sources, namely, non-real-time traffic. Based on presented tests, it is possible to identify that the proposed solution shows solid CPU activation latency results (10 μs), even under different types of concurrent load (CPU, I/O, HDD, network). Also, considering all the test cases, the maximum latencies did not exceed 6 μs, nor did the standard deviation overcome 1.1 μs. A direct comparison between Xenomai and PREEMPT_RT approach is also presented by the authors, and it concluded that the first presents an improvement of at least 30% over the latter. The outcome of this work, namely, when looking at parameters such as latency, overhead, and isolation, is highly encouraging and seems to show that the combination of Xenomai co-kernel approaches and containers can support RT-compliant environments.

5.3 Standalone Kernels Combined with RT Co-kernels

Tasci et al. [76] tried to combine the advantages of standalone kernels and RT co-kernels. This mixed approach still relies on a co-kernel but also explores the potential of single kernels by

applying a real-time patch to the "main" kernel and then using a second kernel for tasks with tighter requirements. The authors proposed an architecture built on this concept that uses containers to modularize RT control applications, with a view towards improving reusability, portability, and flexibility. The core concept relies on distinct modules implemented as Docker containers running a merge between PREEMPT_RT kernel patch and Xenomai Cobalt kernel, which communicate between themselves through a brokerless messaging system based on ZeroMQ framework. It is assumed that each module is implemented in a different container and that multiple containers are needed during the control runtime execution. By combining PREEMPT_RT with the Cobalt Kernel, the authors conclude that it is possible to run multiple applications inside containers in a predictable and deterministic way. While the PREEMPT_RT patched kernel is used for less strict tasks (e.g., tasks that manage and launch real-time tasks), the Cobalt kernel is used for more demanding RT tasks. The outcome is a double kernel system where both kernels are more deterministic than a general-purpose one and both can execute real-time tasks. However, the authors also mentioned that, at that time, the cobalt source needed some fixes to be able to support containers. Moreover, the merge of both patches (Cobalt and PREEMPT_RT) required fixing several file conflicts. The authors' evaluation focused mainly on the round-trip time of messages exchanged between containers, which were observed to be between 50 and 150 μs. Based on this, authors conclude that applications with cyclic times of 500 μs can be successfully executed in this architecture.

5.4 Scheduling-based Methods

As mentioned before, running containers are just processes being executed by the Linux kernel and, as such, are given a processing time slot by the host OS scheduler at a certain time. Therefore, tweaking the way the scheduler works is one of the most relevant aspects for guaranteeing deterministic behavior and achieving RT requirements. Although the previously mentioned works also (partially) rely on scheduler optimizations, in this subsection, we specifically discuss the works focused on the scheduling mechanism itself, instead of other kernel changes or patches.

Telschig et al. [77] presented a cross-domain and cross-platform RT container architecture for dependable and reliable distributed embedded applications for leveraging safe dynamic updates of distributed embedded applications. The logical execution time paradigm is the basis for the proposed architecture, to uncouple functionality from timing, functionality from composition, and domain from platform concerns. Thus, it enables the easier deployment of interacting RT tasks to distributed nodes. The SCHED_DEADLINE scheduler mechanism—which combines the **Constant Bandwidth Server (CBS)** algorithm to compute the scheduling deadlines and the EDF algorithm to schedule the tasks—is used to achieve temporal isolation and to partially guarantee RT performance. According to the authors, this architecture enables a mixed-criticality approach where tasks ranging from hard real-time to best-effort criticality levels can co-exist. Also, for being cross-platform, dependency conflicts can be avoided, allowing the use of legacy components and reducing interoperability issues. A small embedded system combined with LXC containers is described as a working example. However, little or no empirical data retrieved from this example is presented.

Abeni et al. [1] presented a new hierarchical real-time scheduling system for Linux that provides temporal scheduling guarantees for multiple co-located containers while being compatible with the most well-known container-based virtualization solutions. The proposed approach resorts to a 2-level scheduling hierarchy: At the first level, the SCHED_DEADLINE scheduling policy is used, implementing a CBS algorithm, while at the second level, a standard fixed priority scheduler (SCHED_FIFO or SCHED_RR) is used. The former is used to schedule the real-time run queues of the control groups (cgroup), and the latter schedules the real-time tasks inside the cgroups. Experimental results combining LXC containers, the presented scheduling mechanism, and an audio

pipeline show that the systems can achieve better response times and may decrease the needed real-time computational bandwidth, thus reducing or even eliminating the occurrence of xruns (buffer under or overrun events caused by deadline misses).

Likewise, Cucinotta et al. [15, 16] also implement a custom Hierarchical CBS scheduling policy to guarantee RT processing performance within containers located in private clouds, thus guaranteeing a stable performance in distributed cloud services. Similar to the approaches presented above, a SCHED_DEADLINE policy with CBS algorithm is used to select a control group to be scheduled on each CPU, and the SCHED_FIFO or SCHED_RR policy is used to select the tasks in the scheduled control group. This mechanism is presented as a solution to achieve fine-grained control of the temporal interference between co-located real-time services while avoiding overheads incompatible with real-time requirements. LXC containers are used in the validation scenario, which includes multiple **virtualized network functions (VNFs)** deployed as containers, across multiple heterogeneous computing nodes, and with distinct timing requirements. Each container within a node may have its custom scheduling parameter values (Q,P) for a proposed Hierarchical **CBS (Constant Bandwidth Server)** scheduler that extends the Linux kernel SCHED_DEADLINE_CPU scheduler class—(Q,P) corresponds to the amount of runtime (Q) every period (P), meaning that Q time units are granted to a task on the CPU(s) every P time units.

A probabilistic model is presented to optimize such values according to a pattern of requests modelled as a Poisson stochastic process. Various scheduling reservation parameter configurations are tested and compared with theoretical expectations. The empirical results confirm that the presented model can deliver high predictability under the correct reservation parameters, even with interfering tasks.

Struhàr et al. [75] also resort to the use of a Hierarchical Scheduling mechanism and, after an evaluation, conclude that real-time containers running on hosts with such scheduler mechanism are able to keep their allocated resources even in the presence of other RT or best-effort containers running heavy processing loads. Also, it is noticed that the runtime jitter stays very low, thus not influencing the real-time containers but reducing the CPU utilization used by the best-effort containers. Nothing else is mentioned about guaranteeing in-node RT performance, since the authors mainly focus on the orchestration mechanism.

Lee et al. [52] addressed the functionality and QoS issues that may arise when using traditional fieldbuses, such as CAN, in containerized virtual environments. For this matter, a lightweight CAN virtualization technology for containerized controllers is presented. Regarding functionality, a driver-level virtualization technology provides the needed abstractions for the virtual CAN interfaces and buses while still maintaining transparency to the OS and other applications. This supports the sharing of the network interface among multiple virtual controllers in an isolated way while maintaining CAN requirements and low overheads. Regarding the QoS, a hierarchical RT scheduler based on a periodic execution model is used to guarantee the accomplishment of hard RT tasks. Also, a simulator enabling the adjustment of phase offsets of virtual controllers and tasks is provided. This enables sub-optimal phasing of controllers and tasks by taking into consideration a global clock based on the IEEE 1588 standard and the best moment to execute a task, according to the end-to-end delay. Authors claim the presented solution reduced the worst-case end-to-end delay on a real system by up to 78.7%. Unfortunately, no detailed information is provided about the adopted containerized solution.

Similarly to the previous methods, scheduling-based approaches also seem to provide adequate support for real-time operation with worst-case controlled latency requirements in the order of the hundreds of milliseconds. Again, the diversity of the studies, configurations, architectures, and the different approaches adopted in each case makes it difficult to compare different works and infer more detailed conclusions. However, it has been shown that different strategies can be

used while adopting a scheduling-based method, such as 1- or 2-level scheduling hierarchies or distinct scheduling algorithms, to achieve temporal isolation and achieve RT performance even in the presence of concurrent best-effort workload.

5.5 Other Notable Approaches

It was found that several papers did not fit into any of the previous categories due to different factors, such as failing to detail what types of strategies were being used to guarantee real-time performance. This subsection is devoted for such cases that still hold relevance for our research question, discussed next.

Garcia et al. [32] presented a flexible and lightweight container architecture for industrial control to deploy flexible functions and virtualized control units with legacy support. The intended goal is to be able to adapt and deploy distinct distributed services within the system, according to the current needs and configuration. Such a mechanism must take into consideration the strong isolation needs of each service and its associated QoS requirements, such as RT capabilities, security, and safety. The presented architecture relies on the Docker engine for the execution of containers that have a FORTE runtime application inside, developed according to IEC 61499 function block principles. This application is responsible for the message processing and exchange with the cyber-physical systems (robotic arms) using TCP sockets. The **Robot Operating System (ROS)** is used in the nodes that host the containers. Also, EtherCAT drivers are embedded in the system. The conjunction of such technologies seems quite interesting in this context and, presumably, the coexistence of these elements ensures RT requirements when executing rt-tasks. However, nothing is explicitly mentioned about RT guarantees.

Hofer et al. [42] aimed for an Infrastructure-as-a-Service approach, studying the possibility of migrating real-time industrial control applications from dedicated hardware to virtualized servers with shared resources. Although most of the presented work is executed using a type-1 hypervisor (instead of a containerized environment), the presented study and its outcomes are still worth mentioning, as it also encompasses a short review of container frameworks, considering LXC/LXD, Docker, and Balena. A review of operating systems is also presented, including resinOS, Ubuntu Core, Xenomai 3, and PREEMPT_RT, with the last two being selected for comparison. Exhaustive testing is performed under 12 different host configurations, and the outcome regarding latency performance is measured. Then, a hardware comparison was executed using the most favorable configuration previously identified, comparing CPU latency under multiple hardware options made available by **Amazon Web Services (AWS)**. In this comparison, PREEMPT_RT always outperformed XENOMAI 3. The maximum latency achieved was 49 ms and only 96 occurrences out of 10 million (0.00096%) were registered above the predefined upper limit of 100 ms. The best performance by PREEMPT_RT resulted in a considerable low spread and in a peak value of 114 μs). Finally, a Balena container was used for latency tests also executed in the AWS infrastructure. Average values of 11.44 µs (σ0.71 µs) with maximum peaks of 11,644 μs were observed. The results led the authors to conclude it is feasible to migrate RT industrial control applications to virtualized environments with shared resources.

De Simone et al. [17] discuss a lightweight virtualization approach leveraged by hardware-assisted **Trusted Execution Environments (TEE)**, such as the ARM TrustZone technology or the Intel **Software Guard eXtension (SGX)**, to guarantee temporal, spatial, and fault isolation—thus increasing the determinism and safety of the system. This type of technology (internal to the CPU perimeter) provides capabilities of securing user space applications without requiring any call to privileged OS code. Even though it is an interesting approach and the authors briefly analyze it from a containers perspective, they opt to complement the TEE technology with unikernels instead of containers—therefore, most of the presented discussion stays out of the scope of this

article. Nevertheless, such type of technology used along with containers could be quite relevant and worth being aware of.

5.6 Summary

After analyzing the selected works, it can be concluded that, for some authors, guaranteeing real-time performance in containerized environments can be achieved by applying scheduling, priority, and/or process preemption tweaks on Linux systems. However, other authors mention the need to complement those tweaks with custom configurations and the use of specific container platform flags to provide appropriate guarantees for containerized RT tasks.

Despite the diversity of the proposed approaches, it is often difficult to narrow down the specific aspects of each technique or its implementation details. In fact, authors often fail to mention which classes of real-time systems (soft, firm, or hard) were being considered within the scope of their work. Moreover, several works omitted details describing how RT-compliant behavior was implemented/obtained for containerized RT tasks. An example of such omissions can be found in Reference [10], where a Docker container platform was used to support an attack-resilient drone control mechanism running on a Raspberry Pi 3 Model B, which only mentions that a real-time patch was applied to a Linux 4.4 without further information or any evidence regarding the support for such hard real-time system (which is left for future work). In fact, even some studies that provide details about the implementation specifics for containerized RT-capabilities do not always describe in detail how the adopted approach helped ensure compliance with RT requirements. Such omissions make it difficult to replicate the content presented in these works.

Nevertheless, and despite the aforementioned shortcomings, the works hereby discussed confirm that multiple methods can be used to achieve the determinism and predictability targets deemed necessary to comply with RT deadlines, and that these can be used independently or in conjunction with each other. As seen in Table 7, most of the works analyzed in this survey opted for using the PREEMPT_RT approach. This may be due to the following reasons: the wide compatibility across Linux distributions, since it supports every long-term stable version of the mainline Linux kernel since kernel version v2.6.11; because its installation process is quite straightforward; and/or because it demands no extra tweaks or adaptations to the application to be executed (unlike co-kernel approaches that require the use of their own APIs).

6 EXPECTED ORDER OF MAGNITUDE FOR RT TASK LATENCY WHEN USING CONTAINERS

As mentioned before, low latency and real-time requirements are different concepts. By definition, a real-time system can accommodate diversified latency requirements, which can be by the order of magnitude of seconds, minutes, or even higher. However, real-time cyber-physical systems often require stable low latency, or even ultra-low latency response times, so the system can fulfill its intended function (cf. Table 1). However, when it comes to containerized environments it was not possible to systematically assess which latency requirements can be guaranteed. Two main reasons concur for this:

- The techniques presented and the focus of each work are variable, which makes each authors' focus on different aspects of real-time container virtualization and, therefore, present distinct depth according to their focus.
- The results presented in the analyzed papers do not follow any sort of measurement standard. The way of measuring latency differs, as well as the richness of the provided data (e.g., number of measurements performed, maximum value/outliers measured, absence of standard deviation, type of latency measured).

Table 7. Research Summary

Work	RT approach	Latency	RT type	Platform	Orchestration	HW architecture	Focus
Abeni et al. [1]	Scheduling-based (SCHED_DEADLINE extension)	290 μs - 10 ms	Soft-RT, Firm-RT	LXC	N.A.	Intel x86-64	Scheduling
Albanese et al. [2]	Single Kernel-based (PREEMPT-RT)	189 μs - 599 μs	Soft-RT, Firm-RT	Docker	N.A.	Intel x86-64	Processing Latency; Networking latency
Barletta et al. [5]	Co-kernel-based (Xenomai)	10 μs (+-1 μs)	Hard-RT	Docker	N.A.	Intel x86-64	Architecture; Scheduler; Processing Latency; Networking Latency
Carvalho et al. [8] (later detailed in [7])	Single Kernel-based (PREEMPT-RT)	20 μs - 430 μs	Soft-RT, Firm-RT	Docker	TOSCA	Intel x86-64	Network Latency; Processing Latency
Cervini et al. [9]	N.A.	20 ms	Soft-RT	Docker	N.A.	ARM64	Architecture; Resilience
Chen et al. [10]	Linux with a real-time patch	N.A.	Hard-RT	Docker	N.A.	ARM64	Security
Cinque and Cotroneo [12]	Co-kernel-based (RTAI)	N.A.	N.A.	Docker	N.A.	N.A.	Architecture; Isolation; Safety
Cinque and De Tommasi [13]	Co-kernel-based (RTAI)	N.A.	N.A.	Docker; LXC	N.A.	N.A.	Architecture; Isolation; Safety
Cinque et al. [11]	Co-kernel-based (RTAI)	N.A.	Hard-RT	Docker	N.A.	Intel x86-64	Architecture; Processing Latency; Scheduler
Cucinotta et al. [15]	Scheduling-based - SCHED DEADLINE; CBS+SCHED_FIFO; SCHED_RR	N.A.	Hard-RT	LXC	N.A.	Intel x86-64	Processing Latency; Scheduler
Cucinotta et al. [16]	Scheduling-based - SCHED_DEADLINE; CBS+SCHED_FIFO; SCHED_RR	N.A.	Hard-RT	LXC	N.A.	N.A.	Processing Latency; Scheduler
De Simone and Mazzeo [17]	Hardware-assisted Trusted Execution Environments	N.A.	N.A.	N.A.	N.A.	Intel x86-64	Isolation; TEE; Unikernel
Garcia et al. [32]	N.A.	N.A.	N.A.	Docker + FORTE runtime	N.A.	Intel x86-64	Architecture; Function deployment
Goldschmidt and Hauck-Stattelmann [33]	Single Kernel-based (PREEMPT-RT)	100 ms - 1 s (+-10%)	Soft-RT, Firm-RT	LXC; Docker	N.A.	ARM32 Intel x64	Architecture; Legacy support; Function deployment
Goldschmidt et al. [34]	Single kernel-based (PREEMPT-RT)	100 ms - 1 s (+-10%)	Soft-RT, Firm-RT	LXC; Docker	N.A.	ARM32 Intel x86-64	Architecture; Legacy support; Function deployment
Govindaraj and Artemenko [36]	N.A.	N.A.	N.A.	LXC; LXD	Custom	N.A.	Live Migration; Downtime
Hinze et al. [40]	Single Kernel-based (PREEMPT-RT)	4.12 μs (+-0.4102 μs)	N.A.	Docker	N.A.	AMD64	Architecture; Simulation
Hofer et al. [42]	Single and Co-kernel-based (PREEMPT-RT, Xenomai Cobalt)	11.44 μs (+-0.71 μs)	Hard-RT	Balena	N.A.	Intel x86-64	Architecture; Orchestration
Hofer et al. [41]	Single Kernel-based (PREEMPT-RT)	100 ms (+- 126 μs)	Hard-RT	Docker	yes	Intel x86-64	Architecture; Orchestration
Kaur et al. [47]	N.A.	N.A.	N.A.	N.A.	Kubernetes (extended)	N.A.	Orchestration; Scheduler; Green energy

(Continued)

Table 7. Continued

Work	RT approach	Latency	RT type	Platform	Orchestration	HW architecture	Focus
Krüger et al. [50]	N.A.	N.A.	N.A.	Docker	Docker Swarm	ARM	Orchestration; Resource Allocation; Safety
Lee et al. [52]	Scheduling-based - Hierarchical RT scheduler based on periodic execution model	N.A.	Hard-RT	N.A.	N.A.	Intel x86-64	Fieldbus; CAN virtualization; Networking Latency
Li et al. [53]	N.A.	up to 500% more over non-virtualized	N.A.	Docker	N.A.	Intel x86-64	Performance: VMs vs. Containers
Lin et al. [54]	N.A.	N.A.	N.A.	N.A.	N.A.	N.A.	Architecture; Orchestration; Scheduling
Liu et al. [56]	N.A.	200 ms (RTT)	N.A.	Azure IoT Edge (The Moby project)	N.A.	ARM64 Intel x86-64	Performance evaluation of popular edge-cloud architectures
Masek et al. [59]	Single Kernel-based (PREEMPT-RT)	60 - 7,000 μs	Firm-RT, Hard-RT	Docker	N.A.	Intel x86-64	Processing Latency
Moga et al. [61]	Single Kernel-based (PREEMPT-RT)	250 μs - 1 ms	N.A.	Docker	N.A.	Intel x86-64	Virtual Network Latency; Processing Latency
Morabito [62]	N.A.	N.A.	N.A.	Docker	N.A.	ARM32 Intel x86-64	Performance evaluation of popular edge devices
Okwuibe et al. [64]	N.A.	N.A.	N.A.	Docker	Kubernetes	Intel x86-64	Orchestration; Power Consumption; Networking Latency; Migration Latency
Singh et al. [71]	N.A.	N.A.	N.A.	Docker	Docker Swarm	Intel x86-64	Orchestration; Scheduler; Security
Sollfrank et al. [73]	Single Kernel-based (PREEMPT-RT)	117 μs - 513 μs	Hard-RT	Docker	N.A.	ARM64	Network Latency; Processing Latency
Sollfrank et al. [72]	Single Kernel-based (PREEMPT-RT)	242 μs - 513 μs	Soft-RT	Docker	N.A.	ARM64	Network Latency; Processing Latency
Struhàr et al. [75]	Scheduling-based (Hierarchical Scheduling Patch)	100 ms (RTT)	Hard-RT	Docker	Kubernetes (extended)	Intel x86-64	Orchestration; Scheduler
Tasci et al. [76]	Mixed-based (PREEMPT-RT + Xenomai Cobalt)	500 μs	N.A.	Docker	N.A.	N.A.	Architecture; Scheduling
Telschig et al. [77]	Scheduling-based (SCHED DEADLINE: CBS+EDF)	250 ms	Soft-RT, Firm-RT, Hard-RT	LXC	N.A.	Intel x86	Architecture; Scheduling; Safety
Yadav et al. [79]	N.A.	N.A.	N.A.	Docker	Docker Swarm	N.A.	Orchestration; Scheduling; Scaling; Resource Allocation
Yin et al. [80]	N.A.	N.A.	Hard-RT	Docker	N.A.	ARM64 Intel x86-64	Scheduling; Task delay; Resource Allocation

*N.A. - not applicable or not mentioned.

This means that the results presented across different papers cannot be directly compared, nor do the aforementioned asymmetries allow to establish a correlation between latency values and methods used. For example, References [72, 73] were among the very few that presented latency values considering processing latency and network latency, standard deviation and peak values.

Nonetheless, these papers do allow the extraction of latency-indicative values that can be used to gauge the order of magnitude that can be expected when using containers.

6.1 Container-induced Overhead

When it comes to comparing the overhead induced by using containers with a non-virtualized environment, some works [33, 34] conclude that the containerized execution of control applications suffers a negligible and almost constant overhead (around 2 μs) to 5 μs)), with a noticeable increase in determinism, meaning that the latency distribution is narrow. In Reference [59] it is also concluded that the usage of containers has a negligible impact on the system, since no deadline has been missed, and the extra cost of using virtualization is considered to be around 20 μs). Liu et al. [56] also state that container-based virtualization does not add a perceptible performance loss in computing or in communication. However, the analysis performed is situated in the millisecond range, and no in-node real-time method is being used. In Reference [72] it is stated that there is a global increase of latency in containerized environments, namely, more 150 μs in the Ethernet network request-reply with a wider standard deviation (44 μs), and more 144 μs in the on-node processing (with a wider standard deviation around 46 μs). Sollfrank et al. [73] noticed an additional in-node processing latency of 43 μs when using containers. Li et al. [53] highlight the lightweightness of the container's virtualization technology, but also state that the container's performance variability overhead can go as high as 500% over the non-virtualized environment. Also, it is stressed that the performance overhead can vary, depending not only on a feature-to-feature basis but also on a job-to-job basis.

6.2 Overall Latency of Containerized Environments

Regarding the overall latency values measured when using containerized environments, by looking at Table 7 it is possible to observe that some works [2, 5, 40, 61, 72, 73, 76] measured latency values below 1 ms. These seem to be the best-case scenarios. However, others [33, 41, 77] already place the latency values over 100 ms. Nonetheless, it is important to look at these results with close attention, since those are mostly mean values. One should also consider the standard deviation to understand the determinism of the system, and also the peak values to know what is the worst-case scenario. If a soft or even a firm RT system can cohabit with some missed deadlines, then the same does not happen with hard RT systems. Especially in the latter, peak values, rather than average values, should be taken into account when designing the system, so RT requirements are met. For example, in Reference [42] it is reported an average value of 11.44 μs with a standard deviation of 0.71 μs, but peaks of 11,644 μs were observed. Looking only at the first two values, one could conclude that any system with deadlines around 1 ms and allowing jitter up to 1 μs (motion control according to Table 1) could easily be supported. However, the observed peaks raise a red flag for the use of the presented solution for a hard RT system, which has a deadline far below the observed peak. Nevertheless, it could be possible to use such a solution for more flexible soft or firm real-time systems able to tolerate a few missed deadlines.

Summing up, the observed latency results seem to be in line with the requirements of several types of cyber-physical systems. When looking to the processing latency, the cost of using a containerized solution seems to be somewhere between 2 μs and 144 μs. Some authors report more stable values with lower jitter, while others report higher variance in the results. Regardless, there seems to be a gap in the literature in what relates to these types of measurements. A formal procedure for rigorously defining how to measure the different types of latency would be a valuable asset, which could leverage not only the quality of each study but also the comparative analysis between different studies. Struhàr et al. [75] tried to address this gap by studying operating system-level metrics, as well as metrics to specifically evaluate the timeliness of tasks running in

the system, and adapted those to assess the RT performance of containers. Nevertheless, there is still a strong need for formal and validated metrics.

7 CONTAINER PLATFORMS SUITABLE FOR RT SYSTEMS

In general, selected studies use one of the following three distinct container platforms: LXC, Docker, and Balena.

7.1 LXC

LXC (Linux Containers) [57] is a containerization engine that was first presented in 2008 as a contender to virtual machines. Its goal was to enable a virtualization environment with a lower footprint and less overhead when compared to virtual machines. It provides an API and some tools that allow for users to create and manage system or application containers (although being focused on the first), also supporting multiple types of network configurations. System containers tend to be stateful, albeit this is not mandatory, since it basically depends on its intended use. LXC functionalities can be extended with LXD [6], which builds on top of LXC while providing support for distinct network types and better storage configuration. LXD supports the management of multiple instances (not only containers but also virtual machines), also providing functionalities such as running container snapshots, an API designed to leverage third-party tool integration, or simplified over-the-network control, among others.

7.2 Docker

Docker [45] was presented in 2013. This framework, which initially enabled the creation and management of containers based on LXC, has evolved a lot since then. With the release of version 0.9, Docker stopped using LXC as the default engine and replaced it with its own libcontainer, a containerization library natively created in GOlang. This created an abstraction layer that enabled the support of a broader range of isolation techniques and also allowed a considerable reduction of dependencies (since it allowed Docker to control several functionalities without relying on external packages, such as those related to control groups, namespaces, AppArmor profiles, firewall, and network interfaces). It also opened the doors for the use of containers on top of other operating systems and to the OCI standards. Since then, Docker has specialized in application containers. These were originally created to be stateless, ephemeral, portable, and as lightweight as possible. However, just like system containers, it really depends on the usage and on the tradeoff between the aforementioned characteristics and having persistent storage. Like LXC, Docker also supports multiple types of network configuration. It also offers a REST API to simplify the management and integration with third-party tools, although Docker offers a large set of tools such as Docker Swarm, which enables the creation, management, and scheduling of Docker engines and containers.

7.3 Balena

Balena [4], originally designated as Resin.io, is a container-based platform for deploying and managing IoT fleets that started in 2013. Balena's goal was to simplify the development, deployment, and management of software for IoT devices by leveraging the usage of Linux containers and other open technologies. Nowadays, Balena is a full infrastructure ecosystem (balenaCloud) composed of several modules that focus on the needs of IoT fleet owners. One of the most important modules is the container engine (balenaEngine), which is built on top of the Moby Project [66], compatible with Docker containers. It is optimized to run in the edge of IoT environments and, for this reason, it was stripped from capabilities present in Docker that were considered unnecessary in such environments (e.g., Docker Swarm, plugin support, cloud logging drivers, overlay networking

drivers, non-boltdb backed stores as consul, zookeeper, etc.). Also, it uses bandwidth more efficiently, has smaller binaries, and uses storage and RAM more conservatively, to be compatible with less powerful embedded devices. Another relevant module is the balenaOS, which is a lightweight operating system, based on Yocto Linux, which is optimized to run containers on embedded devices with a special focus on reliability over long periods of operation.

7.4 Platform Suitability

The analyzed papers showcase Docker as the dominant platform, with a noticeably wider acceptance. However, this does not mean it might be the best option for all cases. For instance, LXC can be used to leverage system containers to emulate legacy operating systems (as seen in Reference [33], where it was used to emulate an old PowerPC) or when there is a need to provide independent Linux servers while avoiding the cost of using virtual machines or dedicated hardware.

However, Docker is more suitable for micro-services, since it focuses on application containers that are more lightweight, simple to manage and scale. The use of micro-services seems to be the approach that most authors adopt when considering RT environments in IACS. Works such as References [32] and [34] leverage these capabilities, which are deemed suitable for micro-services, to implement the concepts presented in the IEC 61499 standard, namely, the need to work with modular function blocks with granular functionalities and a high potential for reusability.

The aforementioned paper analysis also revealed Docker as an effective option for low-power devices, such as those commonly found at the network edge. An example of this trend is provided by Morabito [62], who studied the use of Docker containers in low-power nodes such as single-board computers. After analyzing the CPU, memory, disk I/O, network performance, and the energy efficiency, among other parameters, it was concluded that there was a negligible impact when using containerized environments in such hardware, even when running multiple concurrent instances. Adding to the size of the Docker community and its extensive documentation, this can explain Docker being the most-used container platform in this area of research.

The standards discussed in Section 2 strongly contributed to the emergence of new solutions to implement container virtualization. This contributed not only to the evolution of existing platforms but also to increasing the compatibility among them. Since both LXC and Docker platforms support images compatible with the OCI standards, it is possible to create an LXC container from a Docker container image. It is also viable to combine both types of containerization. For example, it is possible to provide a complete Linux system using an LXC container and run multiple Docker containers inside, thus taking advantage of a hypothetical hardware consolidation but without having the overhead of a virtual machine and still taking the most of Docker capabilities (such as portability, scalability, and small fingerprint) to run multiple applications or micro-services.

Although Balena was only mentioned in one paper, its optimization for embedded devices, support for IoT fleet management, and the strong reliability provided by Moby makes it a container platform worth considering for IoT environments. In Reference [42] the authors adopted Balena due to its flexibility, ease of use, and the properties coming with the stateless containers.

Overall, the conclusions reached by the different authors led us to believe that, under the correct environment and configurations, most, if not all, of these platforms can deliver deterministic real-time performance.

8 CONTAINER ORCHESTRATION IN RT CONTEXTS

In the world of IACS, we can find systems with varying degrees of complexity, ranging from simple deployments often involving few sensors/actuators and a single control device, to complex and large-scale systems made up of tens, hundreds, or even thousands of devices. For the latter

cases, the adoption of containerization can lead to the creation of a considerable number of container instances, thus becoming potential candidates for the adoption of micro-service based approaches (where a single monolithic service may be partitioned into several micro-services, each running in a distinct container). In such situations, it becomes impracticable to individually manage each container, thus requiring the adoption of container orchestration technologies to address the operational needs of a container fleet often spread across heterogeneous computing nodes. This technology leverages the use of containers by enabling a centralized view of all the container infrastructure resource pool and by allowing the automation of multiple distinct tasks (cf. Section 2). Also, orchestration can be applied in more distinct environments and architectures, such as local, cloud, and edge-cloud. As such, container orchestration encompasses several capabilities and can be optimized and exploited in different ways—as discussed in some of the reviewed works.

8.1 Energy Consumption

In the scope of energy consumption, Kaur et al. [47] tackled the need for scheduling jobs around different IIoT nodes in a containerized network while optimizing energy consumption vs. performance, with minimal interference from coexisting containers. To achieve such goals, a scalable and comprehensive new controller for Kubernetes is presented, whose aim is to map/schedule containers across the available nodes. In this case, orchestration is tackled as a multi-objective optimization problem, formulated with different constraints (i.e., task deadlines, available resources, energy consumption, source of energy, and statistical data). After evaluating the proposed scheduler against existing schedulers using RT Google traces, it is concluded that it improves energy utilization by 14.42%, while also improving the performance/interference ratio by 31.83% and reducing the carbon emissions by 47%, when compared to the FCFS scheduler.

Still in the same scope, Okwuibe et al. [64] researched end-device energy consumption and latency in dynamic environments when combining containerized environments with other technologies such as software-defined networking, network-function virtualization, and multi-access edge computing 5G services. Authors used Kubernetes as the orchestration tool and focused on optimizing the resource allocation for edge-co-located IIoT services. Those services were dynamically offloaded to other nodes closer to the end device, thus helping to reduce the power consumption and latency of end devices. Although the mean migration time was 4.450 ms, the authors suggest it would be possible to reduce this time by applying machine learning techniques and preemptively moving containers ahead of time. It was also concluded that the use of Docker containers increased the energy consumption by up to 4.5% and added 5 seconds to the initialization time of the case-study application.

Other proposals such as Lin et al. [55] propose strategies for VM placement and reallocation aiming at optimizing performance and energy conservation in server clusters. While not specifically oriented towards RT workloads, the **Peak Efficiency Aware Scheduling (PEAS)** could be adapted for container environments, considering RT-compliance in addition to CPU, memory, disk space, and bandwidth constraints, eventually expanding the Computing Resource Unit metric to also cover determinism guarantees.

8.2 Orchestration

Orchestration can also help minimize the cost of executing time-critical containerized tasks in a secure manner. An example for this use case is provided by Singh et al. [71], who used Docker Swarm to enforce security, deadline, and cost constraints. By using game theory, the authors were able to take into account different parameters, such as the cost of the security mechanisms, the resource costs, and the execution deadline thresholds, to schedule the tasks (or a portion of them) along different machines (working nodes). Although some simplifications were made, such as assuming

that the working nodes cannot deny the execution of an assigned fraction or the nonexistence of any sort of dependence between tasks, the validation outcomes seem encouraging.

Orchestration can also help allocate or deallocate resources at the right moment according to the intended goals, something that might be of utmost importance when dealing with time-critical requirements. For instance, Struhár et al. [75] extended the Kubernetes scheduler, optimizing it for RT container scheduling. The implementation consists of two main components: RT Scheduler Extender and the RT Manager. The former extends the Kubernetes control plane, providing admission control and scheduling of real-time containers across the available container nodes. The latter lives on the compute nodes, enabling deployment of real-time containers and monitoring of the in-node performance, which is periodically reported to the master node. Container-level metrics are defined to evaluate the RT performance of tasks being executed on the container nodes: the number of missed deadlines, maximum lateness, and maximum response time within a certain period. Both compute nodes and containers' characteristics and requirements are registered in advance and, if needed, updated. When there is a need to run a new container, the admission control analyzes if the resources and timing requirements of the containers can be met and allocated in one of the available nodes without negatively affecting already running containers. The presented evaluation shows that a mixed-criticality reality is enabled by the implemented system. Multiple RT containers and best-effort containers can co-exist on a single compute node without negatively affecting the performance of real-time containers.

Yadav et al. [79] addressed the availability and performance of containerized services by proposing a resource provisioning mechanism for managing dynamic and fluctuating workloads. This proactive workload manager uses a modified **PID** algorithm (**Proportional Integral Derivative**—a control theory-based algorithm mechanism), in conjunction with the **HAProxy (High Availability Proxy)** load balancer and Docker Swarm orchestration tool, to perform dynamic resource provisioning according to the response time of the system. A response threshold is predefined for the system. The PID algorithm takes average response times as inputs and calculates the optimal number of containers to achieve the desired threshold. In its dynamic variation, the HAProxy uses a different algorithm to optimize the handling of requests, depending on the number of existing containers. When fewer containers are deployed, round-robin is used, and when the number of containers rises, the least-connection algorithm is applied. In other words, the presented mechanism controls the horizontal scaling (in and out) according to the desired system response time. The evaluation results show clear improvements in the overall response time with a quick convergence to the desired value. Although the presented mechanism only uses a single parameter (response time), the authors suggest that other parameters, such as CPU and memory utilization, might be used to achieve better performance.

Yin et al. [80] use fog nodes to provide extra computational resources near the factory floor to meet hard RT requirements. The authors consider that the fog nodes are located near the terminal devices (other aforementioned studies call them edge nodes but assume the same architecture and proximity). Since these nodes have limited computation, storage, and network resources, authors propose a container-based task-scheduling mechanism algorithm that takes into consideration the real-time requirements of tasks and the high concurrency of fog nodes to determine which tasks need to be scheduled to a fog node and which tasks can run in the cloud. A resource-reallocation mechanism is proposed to maximize the resource utilization of the fog nodes while minimizing the task delays (task completion time). This mechanism calculates the resource quota needed for each task in the subsequent period during the execution phase and, if needed, reallocates the CPU resources in the fog node to maximize the data processing per cycle, always taking into consideration the task deadlines. The output of the reallocation mechanism is used as input to the task-scheduling mechanism that, in this way, becomes aware of the available resources on the fog nodes. It is

concluded that using the reallocation mechanism reduces the tasks' completion time by 10%, and that using the proposed task scheduling allows to increase the number of tasks processed in the fog nodes by 5%.

In a slightly different approach, Lin et al. [54] proposed a containerized solution for efficient digital twin simulation for smart industrial systems, aiming at creating a system that consumes fewer resources than those being commonly used, while still producing trustworthy outputs. To this end, the authors presented a **simulation as a service (SimaaS)** architecture that is able to deliver large-scale models on demand, enabling the creation and deployment of digital twins instances (as well as all the related services) across heterogeneous infrastructure nodes. As stated, since the simulation must be synchronized with the physical system, the cloud, edge nodes, and end devices (that may also be containerized) must be selected in an optimal way. Although the authors do not explicitly mention orchestration (except in one of the schematics), the complete description that is provided implicitly points to the use of containers orchestration, covering the following aspects: large-scale on-demand service; efficient management and collaboration of cloud, edge, and end resources to meet the strict latency requirements; scheduling and automatic deployment of containers; and dynamism requirements. Based on their testing simulation, authors empirically confirm a significant improvement regarding system efficiency, compared to typical heavyweight virtual machine alternatives.

Carvalho et al. [8] (and later in Reference [7]) used OASIS **TOSCA (Topology and Orchestration Specification for Cloud Applications)** templates [65] for orchestration purposes within the scope of a protection and control architecture for electric power substations using containerized real-time services, providing a portable way to define topologies, connections, dependencies, capabilities, and requirements for the entire infrastructure. For the specific implementation, the authors resorted to the xOpera orchestrator [19], together with Ansible playbooks, to define orchestration actuators corresponding to the TOSCA lifecycle standard interface operations for provisioning and configuration automation.

In brief, container orchestration can be applied in very distinct environments, as well as local, cloud, or edge-cloud architectures. In such scenarios, orchestration capabilities can be instrumental for RT-optimized container scheduling, provisioning, and instantiation. As previously mentioned, orchestration capabilities can be used to lower costs by optimizing resource allocation, according to real-time task requirements and fog node availability, thus leveraging the combination of cloud and edge infrastructures to determine which tasks need to be scheduled to a fog node and which tasks can run in the cloud. Such capabilities can be applied in the context of use cases such as digital twin management and deployment or reinforcement of infrastructure security and resilience in scenarios with specific RT requirements (as it is the case for electric power substations in smart grids, for example). As such, the orchestration of containers is one of the most relevant mechanisms to leverage the use of containers in distributed systems.

8.3 Live Migration

Another relevant factor while using orchestration has to do with the ability to perform live container migration. Live migration is the action of moving a container from one server to another without losing the state of the running applications/services (including RAM, CPU, and network state), independently of the server location. This is useful for many situations: to move workloads to other nodes or locations prior to performing server or infrastructural maintenance operations; when there is a need to load balance a system; or when there is a need to keep the services as close as possible to mobile end-devices that are on the move.

When performing a migration, it is essential keep downtime to a minimum. This is a critical point, since during this stage the services running inside the containers become inaccessible. If

the downtime is higher than the response time required by the industrial application, one or more deadlines will be missed and a hazardous result can occur.

Govindaraj et al. [36] proposed a new migration scheme named redundancy migration and evaluated it against the LXC/LXD stock migration. The authors focused especially on edge computing and started by splitting the concept of downtime into migration time and downtime. The former corresponds to the full duration of the migration, starting from the migration trigger until the moment the services become available on the destination server. The latter represents the period during which the services are completely halted. According to the authors, downtime is the more relevant metric, since it used in **service level agreements (SLA)**. The proposed redundancy migration relies on a migration controller and on a traffic controller that runs on both source and destination servers and has four main phases. First, a buffer and a rerouting of traffic between the client and the container are created. Next, a clone of the container is created in the destination server. Next, the new container starts to consume the packets kept in the buffer to catch up to the state of the initial container. Finally, the new container takes over and the initial container can be shut down. Experimental results show a downtime improvement by a factor of 1.8 (from 2.97 s to 1.68 s) and an extra overhead in the migration time by a factor of 1.7 (from 9.00 to 24.3), in comparison to the LXC/LXD stock live migration. Although the migration time is improved, the mechanism that supports it is responsible for increasing the migration time.

Krüger et al. [50] used the dynamic orchestration characteristics offered by Docker Swarm to reallocate containerized services to mitigate the failure of power-grid services caused by disruptive events in the grid services. An RT test platform was created to test the presented solution. The control function of devices such as **Intelligent Electronic Devices (IED)** and **Remote Terminal Units (RTUs)** was virtualized using Docker containers, as well as multiple grid functions and services such as State Estimation, Data Acquisition, and Coordinated Voltage Control. Constant monitoring is performed, and the outcome data is sent to an anomaly detector. When abnormal behavior is detected, an alarm is raised and sent to the Service Controller (Docker Swarm), which then decides if and which services need to be reallocated (it may be only one service or a full service chain of services). It is concluded that this mitigation strategy based on container orchestration reduces the disruption of services.

8.4 Summing Up

The simple use of containers, without extra technology or tools, can bring added value to an IACS by itself. Nevertheless, containerized solutions are greatly empowered when used in conjunction with orchestration tools. This approach assumes a relevant role in all types of scenarios that demand automated actions triggered by information collected from the system infrastructure continuous monitoring and/or other sources, especially in distributed architectures such as edge-cloud or IIoT environments. Failing to launch a container at the right time or at the right location may lead to service unavailability, SLA violations, higher power consumption, higher latency, and other constraints. As such, when planning and designing a containerized environment, one should always take into account the advantages of using orchestration tools to optimize the system and achieve the best performance according to the intended goals.

9 OPEN CHALLENGES

The interest in the use of container virtualization has been growing over the past years, especially in the IT world, prompting its evolution towards increased maturity levels. Part of this popularity is also due to the community that emerged around these technologies and which has been steadily growing up to a sizeable number of contributors willing to address open questions and contribute

to their evolution. The usage of container technology in operational environments with real-time requirements is a different matter, due to the context-specific challenges that are characteristic of such a niche domain. While the previous chapters identified and addressed numerous challenges related to the usage of containers in RT environments (in many cases with quite interesting solutions), there are several relevant gaps to be tackled. Next, we compile some of the more relevant challenges identified by the authors of the surveyed papers.

9.1 Container Placement

When deploying a container image or performing a live migration of real-time containers, it is necessary to pre-allocate resources on the destination host according to the requirements of the tasks running inside the containers. Moreover, it is necessary to be aware that other containers may be running on that same host with their own resource needs and potential concurrency impacts. This process can take some valuable time and may lead to some overhead or even service downtime. Also, it is necessary to be aware of the network status and availability of the destination host.

Yen et al. [80] alerted for the need to address the process of finding an optimal node to place the container to further reduce the task-execution time and network traffic. Cucinotta et al. [15] state there is still work to be done regarding resource allocation to dynamic workloads and on how to deal with overload conditions. In line with this observation, Moga et al. [61] alert to the case where a real-time task does not respect its designed parameters (due to being compromised or other anomalies) and say it is necessary to have a dynamic orchestration algorithm that can deal with such situations. Abeni et al. [1] think there is a need for a more in-depth analysis regarding the scheduler of parallel RT activities deployed in multi-CPU containers. Govindaraj et al. [36] mention that live migration downtime could be further reduced, and Morabito [62] alerts for the lack of support from Docker to perform live container migrations between different entities, especially when there are strict requirements on latency. Okwuibe et al. [64] state that an extra layer to handle orchestration introduces extra latency, and Struhar et al. [75] go further ahead and say that no orchestration system is natively considering RT requirements for containerized applications. Also, Moga et al. [61] state the need for middleware able to host containerized micro-services with RT capabilities that comprise communication, runtime, and isolation.

9.2 Communication

As already mentioned, a containerized service can consist of multiple real-time tasks being executed in several distinct containers, following a pattern similar to micro-service deployments. This necessarily implies that determinism may be important not only for in-node processing, but also for inter-container communications.

Some authors have questioned whether inter-container communications are able to comply with RT requirements [40, 59], with similar doubts being raised regarding data sharing across containers [33, 34]—such concerns are further aggravated by the fact that containers may or may not be hosted in the same node. As such, Tasci et al. [76] stated that container design cannot assume the availability of shared memory/resources for inter-container communication, thus calling for the need of message system frameworks compliant with real-time requirements. The determinism of the standard Ethernet is also questioned by Sollfrank et al. [72, 73], which deems it not suitable for RT environments. This doubt is also expressed by Albanese et al. [2], who suggest further research regarding this matter should be undertaken, for instance, addressing the use of Time Sensitive Networking (that, among other standards, includes the IEEE 802.1AS standard for time synchronization).

9.3 Security and Safety

The specific security and safety concerns for container virtualization have been widely addressed in the IT world in the past few years, with clear improvements. However, in the specific domain of RT applications, only a few works have focused on security and safety aspects.

Cinque et al. [12, 13] raised some questions regarding the isolation of faults in the hosting nodes. There is a need to reinforce the isolation mechanisms that prevent the propagation of faults from the host system to the containers and between containers. If such a problem is not effectively addressed, then a single external fault may compromise the entire system.

Chen et al. [10] focused on defending real-time systems against DoS, achieving interesting results. However, they were not able to validate their approaches on hard real-time systems. In our opinion, there is still work to be done in this area, since hard real-time systems cannot afford missed deadlines, making it more challenging to address security and safety.

9.4 Public Infrastructure

In the IT world, the use of public cloud or edge-cloud infrastructures for containerized solutions is very common, due to factors such as accessibility, convenience, and scalability. However, when deterministic real-time requirements are mandatory, this may not be the best choice. Liu et al. [56] state that current implementations of edge-cloud containerized infrastructures do not fully support the requirements of RT industrial applications, especially in the context of concurrency, and suggest that the performance should be improved up to 20 times.

9.5 Others

Storage transaction speed also seems to be an open issue when using containers in time-restricted systems. While it seems proven that the latency at the processing level can reach values and determinism compatible with RT systems, according to Li et al. [53], the same does not hold in terms of storage transactions' speed, which may represent a bottleneck for systems with I/O needs.

Thinking on the creation of containerized PLC logic, and with the intent of reducing human error, Cervini et al. [9] point to the lack of an automated tool that could ingest logic samples and output containerized PLC variants. Also, such tool should automatically validate the logical equivalence of both physical and containerized PLC and, eventually, propose some optimizations.

10 CONCLUSIONS

In this article, we presented a systematic literature review on container virtualization applied to real-time environments, with a special focus on industrial and automation control systems.

By looking for answers to the proposed research questions, it was possible to identify not only how containers are being used in this context, but also several relevant aspects, such as: the techniques being used to guarantee real-time compliance in the container hosting nodes; the order of magnitude for the in-node processing latency; the container platforms being used and when to choose each; how container orchestration is being applied; and key remaining challenges of container technology in this domain. It is concluded that containerized solutions are compatible with industrial and automation real-time cyber-physical systems. Several works have shown the capability of running containerized real-time tasks in multiple hardware architectures with distinct processing capabilities and in the presence of concurrent workloads. It was also proven that low-powered single-board computers can also comply with RT requirements, opening the door for the possibility of executing a real-time task in the network edge, where the hardware resources tend to be more limited than in cloud architectures or in IIoT systems.

However, there are still some challenges that need to be addressed, especially regarding the deployment and orchestration of containers, and also at networking level. Some research works

partially addressed those challenges with encouraging results. Some validate the usage of full orchestrated containerized solutions, but only in laboratory environments. More research is needed before securely transposing this technology to production environments, especially research that encompasses and validates as a whole all the different aspects of container virtualization (runtime, orchestration, networking).

REFERENCES

[1] Luca Abeni, Alessio Balsini, and Tommaso Cucinotta. 2018. Container-based real-time scheduling in the Linux kernel. *ACM SIGBED Rev.* 16, 3 (Nov. 2018), 33–38. DOI: https://doi.org/10.1145/3373400.3373405

[2] Giuliano Albanese, Robert Birke, Georgia Giannopoulou, Sandro Schönborn, and Thanikesavan Sivanthi. 2021. Evaluation of networking options for containerized deployment of real-time applications. In *26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA'21)*. IEEE, 1–8. DOI: https://doi.org/10.1109/ETFA45728.2021.9613320

[3] Amazon. 2022. Amazon Elastic Kubernetes Service. Retrieved from https://aws.amazon.com/pt/eks

[4] Balena. 2022. Balena. Retrieved from https://www.balena.io

[5] Marco Barletta, Marcello Cinque, Luigi De Simone, and Raffaele Della Corte. 2002. Achieving isolation in mixed-criticality industrial edge systems with real-time containers. In *34th Euromicro Conference on Real-Time Systems (ECRTS'22)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik. DOI: 10.4230/LIPIcs.ECRTS.2022.15

[6] Ltd. Canonical. 2022. What is LXD? Retrieved from https://linuxcontainers.org/lxd/introduction/

[7] Ricardo Carvalho. 2023. *Software Defined Virtualization for Virtual Power Plants*. Master's thesis. University of Aveiro, Department of Electronics, Telecommunications and Informatics.

[8] Ricardo Carvalho, Màrio Antunes, João Paulo Barraca, Diogo Gomes, and Rui L. Aguiar. 2022. Design and evaluation of a low-latency CPC environment for virtual IEDs. In *IEEE 11th International Conference on Cloud Networking (CloudNet'22)*. IEEE, 272–276. DOI: https://doi.org/10.1109/CloudNet55617.2022.9978874

[9] James Cervini, Aviel Rubin, and Lanier Watkins. 2021. A containerization-based backfit approach for industrial control system resiliency. In *IEEE Security and Privacy Workshops (SPW'21)*. IEEE, 246–252. DOI: https://doi.org/10.1109/SPW53761.2021.00043

[10] Jiyang Chen, Zhiwei Feng, Jen-Yang Wen, Bo Liu, and Lui Sha. 2019. A container-based DoS attack-resilient control framework for real-time UAV systems. In *Design, Automation & Test in Europe Conference & Exhibition (DATE'19)*. IEEE, 1222–1227. DOI: https://doi.org/10.23919/DATE.2019.8714888

[11] Marcello Cinque, Raffaele Della Corte, Antonio Eliso, and Antonio Pecchia. 2019. RT-CASEs: Container-based virtualization for temporally separated mixed-criticality task sets. In *31st Euromicro Conference on Real-Time Systems (ECRTS'19) (Leibniz International Proceedings in Informatics (LIPIcs))*, Sophie Quinton (Ed.), Vol. 133. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 5:1–5:22. DOI: https://doi.org/10.4230/LIPIcs.ECRTS.2019.5

[12] Marcello Cinque and Domenico Cotroneo. 2018. Towards lightweight temporal and fault isolation in mixed-criticality systems with real-time containers. In *48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W'18)*. IEEE, 59–60. DOI: https://doi.org/10.1109/DSN-W.2018.00029

[13] Marcello Cinque and Gianmaria De Tommasi. 2017. Work-in-progress: Real-time containers for large-scale mixed-criticality systems. In *IEEE Real-Time Systems Symposium (RTSS'17)*. IEEE, 369–371. DOI: https://doi.org/10.1109/RTSS.2017.00046

[14] Clarivate. 2022. Web of Science. Retrieved from https://www.webofscience.com/

[15] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, and Carlo Vitucci. 2018. Virtual network functions as real-time containers in private clouds. In *IEEE 11th International Conference on Cloud Computing (CLOUD'18)*. IEEE, 916–919. DOI: https://doi.org/10.1109/CLOUD.2018.00135

[16] Tommaso Cucinotta, Luca Abeni, Mauro Marinoni, Alessio Balsini, and Carlo Vitucci. 2019. Reducing temporal interference in private clouds through real-time containers. In *IEEE International Conference on Edge Computing (EDGE'19)*. IEEE, 124–131. DOI: https://doi.org/10.1109/EDGE.2019.00036

[17] Luigi De Simone and Giovanni Mazzeo. 2019. Isolating real-time safety-critical embedded systems via SGX-based lightweight virtualization. In *IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW'19)*. IEEE, 308–313. DOI: https://doi.org/10.1109/ISSREW.2019.00089

[18] Docker. 2022. Container Network Model. Retrieved from https://docs.docker.com/network/

[19] XLAB d.o.o.2023. GitHub - xlab-si/xopera-opera: xOpera Orchestrator Compliant with TOSCA YAML v1.3 in the making. Retrieved from https://github.com/xlab-si/xopera-opera

[20] Michael Eder. 2016. Hypervisor vs container-based virtualization. In *Seminars Future Internet (FI'16) and Innovative Internet Technologies and Mobile Communications (IITM'16)*. Technical University Munich, Munich, DE, 1–7. DOI: https://doi.org/10.2313/NET-2016-07-1_01

[21] Elsevier. 2022. Science Direct. Retrieved from https://www.sciencedirect.com
[22] Elsevier. 2022. Scopus. Retrieved from https://www.scopus.com
[23] Santiago Figueroa-Lorenzo, Javier Añorga, and Saioa Arrizabalaga. 2019. A role-based access control model in modbus SCADA systems. A centralized model approach. *Sensors (Switz.)* 19, 20 (2019), 4455. DOI: https://doi.org/10.3390/s19204455
[24] Arlene Fink. 2020. *Conducting Research Literature Reviews: From the Internet to Paper*. Sage Publications, Washington.
[25] Association for Computing Machinery. 2022. ACM Digital Library. Retrieved from https://dl.acm.org
[26] Cloud Native Computing Foundation. 2022. Cloud Native Computing Foundation. Retrieved from https://www.cncf.io/
[27] Cloud Native Computing Foundation. 2022. Container Network Interface. Retrieved from https://github.com/containernetworking/cni
[28] The Linux Foundation. 2022. Kubernetes. Retrieved from https://kubernetes.io/
[29] The Linux Foundation. 2022. Open Container Initiative. Retrieved from https://opencontainers.org/
[30] The Linux Foundation. 2022. PREEMPT_RT patch. Retrieved from https://wiki.linuxfoundation.org/realtime/preempt_rt_versions
[31] The Linux Foundation. 2022. The Real Time Linux. Retrieved from https://wiki.linuxfoundation.org/realtime/start
[32] Carlos A. Garcia, Marcelo V. Garcia, Edurne Irisarri, Federico Pérez, Marga Marcos, and Elisabet Estevez. 2018. Flexible container platform architecture for industrial robot control. In *IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA'18)*, Vol. 1. IEEE, 1056–1059. DOI: https://doi.org/10.1109/ETFA.2018.8502496
[33] Thomas Goldschmidt and Stefan Hauck-Stattelmann. 2016. Software containers for industrial control. In *42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA'16)*. IEEE, 258–265. DOI: https://doi.org/10.1109/SEAA.2016.23
[34] Thomas Goldschmidt, Stefan Hauck-Stattelmann, Somayeh Malakuti, and Sten Grüner. 2018. Container-based architecture for flexible industrial control applications. *J. Syst. Archit.* 84 (2018), 28–36. DOI: https://doi.org/10.1016/j.sysarc.2018.03.002
[35] Google. 2022. Google Kubernetes Engine. Retrieved from https://cloud.google.com/kubernetes-engine
[36] Keerthana Govindaraj and Alexander Artemenko. 2018. Container live migration for latency critical industrial applications on edge computing. In *IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA'18)*, Vol. 1. IEEE, 83–90. DOI: https://doi.org/10.1109/ETFA.2018.8502659
[37] Jurgen Greifeneder and Georg Frey. 2008. Reactivity analysis of different networked automation system architectures. In *IEEE International Conference on Emerging Technologies and Factory Automation*. IEEE, 1031–1038. DOI: https://doi.org/10.1109/ETFA.2008.4638520
[38] Red Hat. 2022. OpenShift. Retrieved from https://www.redhat.com/en/technologies/cloud-computing/openshift
[39] Red Hat. 2022. Red Hat. Retrieved from https://www.redhat.com/
[40] Christoph Hinze, Timur Tasci, Armin Lechler, and Alexander Verl. 2018. Towards real-time capable simulations with a containerized simulation environment. In *25th International Conference on Mechatronics and Machine Vision in Practice (M2VIP'18)*. IEEE, 1–6. DOI: https://doi.org/10.1109/M2VIP.2018.8600827
[41] Florian Hofer, Martin Sehr, Alberto Sangiovanni-Vincentelli, and Barbara Russo. 2021. Industrial control via application containers: Maintaining determinism in IAAS. *Syst. Eng.* 24, 5 (2021), 352–368. DOI: https://doi.org/10.1002/sys.21590
[42] Florian Hofer, Martin A. Sehr, Antonio Iannopollo, Ines Ugalde, Alberto Sangiovanni-Vincentelli, and Barbara Russo. 2019. Industrial control via application containers: Migrating from bare-metal to IAAS. In *IEEE International Conference on Cloud Computing Technology and Science (CloudCom'19)*. IEEE, 62–69. DOI: https://doi.org/10.1109/CloudCom.2019.00021
[43] IEC. 2012. IEC 61499-1:2012 Function blocks—Part 1: Architecture. Retrieved from https://webstore.iec.ch/publication/5506
[44] IEEE. 2022. IEEE Xplore. Retrieved from https://ieeexplore.ieee.org
[45] Docker Inc.2022. Docker. Retrieved from https://www.docker.com
[46] Open Container Initiative. 2022. Standards Specification. Retrieved from https://github.com/opencontainer
[47] Kuljeet Kaur, Sahil Garg, Georges Kaddoum, Syed Hassan Ahmed, and Mohammed Atiquzzaman. 2020. KEIDS: Kubernetes-based energy and interference driven scheduler for industrial IoT in edge-cloud ecosystem. *IEEE Internet Things J.* 7, 5 (2020), 4228–4237. DOI: https://doi.org/10.1109/JIOT.2019.2939534
[48] Waqas Ali Khan, Lukasz Wisniewski, Dorota Lang, and Jürgen Jasperneite. 2017. Analysis of the requirements for offering Industrie 4.0 applications as a cloud service. In *IEEE 26th International Symposium on Industrial Electronics (ISIE'17)*. IEEE, 1181–1188. DOI: https://doi.org/10.1109/ISIE.2017.8001413

[49] B. Kitchenham and S. Charters. 2007. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE 2007-001, Keele University and Durham University Joint Report.

[50] Carsten Krüger, Anand Narayan, Felipe Castro, Batoul Hage Hassan, Shadi Attarha, Davood Babazadeh, and Sebastian Lehnhoff. 2020. Real-time test platform for enabling grid service virtualisation in cyber physical energy system. In *25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA'20)*, Vol. 1. IEEE, 109–116. DOI: https://doi.org/10.1109/ETFA46521.2020.9211939

[51] Kubernetes. 2022. Container Runtime Interface. Retrieved from https://kubernetes.io/docs/concepts/architecture/cri/

[52] Sang-Hun Lee, Jong-Seo Kim, Jong-Soo Seok, and Hyun-Wook Jin. 2019. Virtualization of industrial real-time networks for containerized controllers. *Sensors* 19, 20 (2019), 4405. DOI: https://doi.org/10.3390/s19204405

[53] Zheng Li, Maria Kihl, Qinghua Lu, and Jens A. Andersson. 2017. Performance overhead comparison between hypervisor and container based virtualization. In *IEEE 31st International Conference on Advanced Information Networking and Applications (AINA'17)*. IEEE, 955–962. DOI: https://doi.org/10.1109/AINA.2017.79

[54] Ting Yu Lin, Guoqiang Shi, Chen Yang, Yingxi Zhang, Jiezhang Wang, Zhengxuan Jia, Liqin Guo, Yingying Xiao, Zhiqiang Wei, and Shulin Lan. 2021. Efficient container virtualization-based digital twin simulation of smart industrial systems. *J. Clean. Product.* 281 (2021), 124443. DOI: https://doi.org/10.1016/j.jclepro.2020.124443

[55] Weiwei Lin, Wentai Wu, and Ligang He. 2022. An on-line virtual machine consolidation strategy for dual improvement in performance and energy conservation of server clusters in cloud data centers. *IEEE Trans. Serv. Comput.* 15, 2 (2022), 766–777. DOI: https://doi.org/10.1109/TSC.2019.2961082

[56] Yu Liu, Dapeng Lan, Zhibo Pang, Magnus Karlsson, and Shaofang Gong. 2021. Performance evaluation of containerization in edge-cloud computing stacks for industrial applications: A client perspective. *IEEE Open J. Industr. Electron. Soc.* 2 (2021), 153–168. DOI: https://doi.org/10.1109/OJIES.2021.3055901

[57] LXC. 2022. Linux Containers. Retrieved from https://www.linuxcontainers.org/

[58] Heather MacKenzie, Ann Dewey, Amy Drahota, Sally Kilburn, P. Kalra, Carole Fogg, and D. Zachariah. 2012. Systematic reviews: What they are, why they are important, and how to get involved. *J. Clinic. Prevent. Cardiol.* 1, 4 (Oct. 2012), 193–202.

[59] Philip Masek, Magnus Thulin, Hugo Andrade, Christian Berger, and Ola Benderius. 2016. Systematic evaluation of sandboxed software deployment for real-time software on the example of a self-driving heavy vehicle. In *IEEE 19th International Conference on Intelligent Transportation Systems (ITSC'16)*. IEEE, 2398–2403. DOI: https://doi.org/10.1109/ITSC.2016.7795942

[60] Microsoft. 2022. Azure Container Instances. Retrieved from https://azure.microsoft.com/en-us/products/container-instances/

[61] Alexandru Moga, Thanikesavan Sivanthi, and Carsten Franke. 2016. OS-level virtualization for industrial automation systems: Are we there yet? In *31st Annual ACM Symposium on Applied Computing (SAC'16)*. Association for Computing Machinery, New York, NY, 1838–1843. DOI: https://doi.org/10.1145/2851613.2851737

[62] Roberto Morabito. 2017. Virtualization on internet of things edge devices with container technologies: A performance evaluation. *IEEE Access* 5 (2017), 8835–8850. DOI: https://doi.org/10.1109/ACCESS.2017.2704444

[63] Springer Nature. 2022. SpringerLink. Retrieved from https://link.springer.com/

[64] Jude Okwuibe, Juuso Haavisto, Erkki Harjula, Ijaz Ahmad, and Mika Ylianttila. 2020. SDN enhanced resource orchestration of containerized edge applications for industrial IoT. *IEEE Access* 8 (2020), 229117–229131. DOI: https://doi.org/10.1109/ACCESS.2020.3045563

[65] OASIS Open. 2014. Topology and Orchestration Specification for Cloud Applications Version 1.0 — docs.oasis-open.org. Retrieved from http://docs.oasis-open.org/tosca/TOSCA/v1.0/TOSCA-v1.0.html

[66] The Moby Project. 2022. The Moby Project. Retrieved from https://mobyproject.org/

[67] Rui Queiroz, Tiago Cruz, and Paulo Simões. 2023. Testing the limits of general-purpose hypervisors for real-time control systems. *Microproc. Microsyst.* 99, 104848 (5 2023). DOI: https://doi.org/10.1016/j.micpro.2023.104848

[68] Federico Reghenzani, Giuseppe Massari, and William Fornaciari. 2019. The real-time Linux kernel: A survey on PREEMPT_RT. *ACM Comput. Surv.* 52, 1, Article 18 (Feb. 2019), 36 pages. DOI: https://doi.org/10.1145/3297714

[69] RTAI. 2021. RTAI - The RealTime Application Interface. Retrieved from https://www.rtai.org/

[70] RTnet. 2012. RTnet. Retrieved from http://www.rtnet.org/

[71] Chitranjan Singh, Preti Kumari, Rahul Mishra, Hari Prabhat Gupta, and Tanima Dutta. 2022. Secure industrial IoT task containerization with deadline constraint: A Stackelberg game approach. *IEEE Trans. Industr. Inform.* 18, 12 (2022), 8674–8681. DOI: https://doi.org/10.1109/TII.2022.3156647

[72] Michael Sollfrank, Frieder Loch, Steef Denteneer, and Birgit Vogel-Heuser. 2021. Evaluating docker for lightweight virtualization of distributed and time-sensitive applications in industrial automation. *IEEE Trans. Industr. Inform.* 17, 5 (2021), 3566–3576. DOI: https://doi.org/10.1109/TII.2020.3022843

[73] Michael Sollfrank, Frieder Loch, and Birgit Vogel-Heuser. 2019. Exploring Docker containers for time-sensitive applications in networked control systems. In *IEEE 17th International Conference on Industrial Informatics (INDIN'19)*, Vol. 1. IEEE, 1760–1765. DOI: https://doi.org/10.1109/INDIN41052.2019.8972165

[74] John Wiley & Sons. 2022. Wiley. Retrieved from https://onlinelibrary.wiley.com
[75] Vàclav Struhàr, Silviu S. Craciunas, Mohammad Ashjaei, Moris Behnam, and Alessandro V. Papadopoulos. 2021. REACT: Enabling real-time container orchestration. In *26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA'21)*. IEEE, 1–8. DOI: https://doi.org/10.1109/ETFA45728.2021.9613685
[76] Timur Tasci, Jan Melcher, and Alexander Verl. 2018. A container-based architecture for real-time control applications. In *IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC'18)*. IEEE, 1–9. DOI: https://doi.org/10.1109/ICE.2018.8436369
[77] Kilian Telschig, Andreas Schönberger, and Alexander Knapp. 2018. A real-time container architecture for dependable distributed embedded applications. In *IEEE 14th International Conference on Automation Science and Engineering (CASE'18)*. IEEE, 1367–1374. DOI: https://doi.org/10.1109/COASE.2018.8560546
[78] Xenomai. 2022. Xenomai. Retrieved from https://source.denx.de/Xenomai/xenomai
[79] Mahendra Yadav, Harishchandra Akarte, and Dharmendra Yadav. 2020. Container elasticity: Based on response time using Docker. *Recent Adv. Comput. Sci. Commun.* 13 (10 2020). DOI: https://doi.org/10.2174/2666255813999201012192010
[80] Luxiu Yin, Juan Luo, and Haibo Luo. 2018. Tasks scheduling and resource allocation in fog computing based on containers for smart manufacturing. *IEEE Trans. Industr. Inform.* 14, 10 (2018), 4712–4721. DOI: https://doi.org/10.1109/TII.2018.2851241

Received 19 December 2022; revised 16 August 2023; accepted 21 August 2023